

9.4 ROBOTIC VISION

The most powerful sensor, which can equip a robot with large variety of sensory information, is robotic vision. Robotic, or synonymously, “computer” or “machine” vision systems are among the most complex sensory systems in use. Robotic vision may be defined as the process of acquiring and extracting information from images of 3-D world.

Robotic vision is primarily targetted at manipulation and interpretation of image and use of this information in robot operation control. Robotic vision requires two aspects to be addressed. One, provision for visual input, and two, processing required to productively utilize the visual information in a computer-based system. The architecture of vision systems, the components for visual input, image representation, and image storage are discussed first. The processing of the acquired image to extract the information from it is discussed later.

9.5 INDUSTRIAL APPLICATIONS OF VISION-CONTROLLED ROBOTIC SYSTEMS

Vision systems can provide information about the position, orientation, identity, and condition of each part in the surroundings. This valuable knowledge can be used to automate the manipulation of objects, plan robot motions to avoid collision with obstacles, or decide how to grasp an object. Although the volume of data produced by a vision system is largest, the amount of information present in an image cannot be matched by any other sensory system. The effective use of robotic vision system makes assembly, quality control, parts handling, and classification tasks more robust.

Using a single camera, it is possible to track multiple objects in visually cluttered environments. A vision-controlled industrial robot can be deployed for a number of different applications.

9.5.1 Presence

The simplest application is to find the presence or absence of a part at a specific location, say on a conveyor belt or in a bin. Many other simple sensors such as proximity or touch sensors can be used for this, but the visual detection of presence combined with other applications discussed next gives much more accurate and versatile information.

9.5.2 Object Location

The parts or obstacles in the visual fields can be located accurately and their position and orientation can be determined with precision. Accurate coordinate assessment is used for tracking motion of manipulator, end-effector, objects, or obstacles. Sometimes, for this purpose special identification markings are made on the parts.

9.5.3 Pick and Place

The manipulator can be guided to pick parts from a specific location after its presence has been detected or from any imaged location in the workcell and place it at the desired location. The gripper can be oriented according to the part's orientation to hold the part properly. To pick a part from a moving conveyor or to put on it is inherently easier than to lift it from a storage bin or place it there.

9.5.4 Object Identification

The objects captured in an image can be identified and distinguished from each other. This may be a part of sorting process, involving determination of location and pick-and-place tasks to effect physical movement of parts.

9.5.5 Visual Inspection

This is a very powerful and potential application area for automated quality control using robots, while they are performing other tasks. Visual inspection is based on extracting specific quantitative measurements of desired parameters from an image.

The robot involved in visual inspection can do sorting of 'good' and 'bad' parts or control the manufacturing process based on measured values of parameters.

9.5.6 Visual Guidance

The image of the scene can be used for accurate specification of relative positions of the manipulator and the part in the scene as well as their relative movements. An example of application of visual guidance is the assembly operation, which requires accurate position and orientation control of two parts to be fitted together. To assemble the parts, one part is held stationary, while the other is maneuvered by the robot, whose movements are controlled by the visual feedback. Another example can be of guiding the motion of the manipulator through stationary or mobile obstacles in the environment of the robot.

Potential industrial applications for vision-controlled robots are numerous and with the help of vision systems robots can be made to perform difficult and complex tasks. From the above applications, it should be clear that the task that can be performed depends on what information is obtained from the image.

The processing hardware and the concepts involved in robotic vision are considerably complex as compared to other sensory systems. The subject of robotic vision is subjective in nature and generalized analytic solutions are yet to be found. Even after years of research, robotic vision remains a challenge. It can be said that the robotic vision is still in the early stages of development but it holds the future as the most powerful robot sensory technology.

9.11 IMAGE PROCESSING

The digitized image is stored in the computer memory for processing. There are two processing objectives: (i) to extract the spatial parameters of the objects and (ii) to recognize the objects in the image. On a 2-D visual image, integral operations can be applied to extract many useful features about the image. Some of these are:

1. Largeness (size or area).
2. Center of area or gravity (location).
3. Inclination (orientation).
4. Axis of least moment of inertia.
5. Other features (shape, geometry, radius etc.)
6. Similarity or dissimilarity (recognition).

Before an image is processed for drawing conclusions, it is usually preprocessed to remove the noise and other imperfections. Improving the quality to facilitate extraction of the desired information is the first part of image processing.

9.11.1 Image Improvement

The image as captured may contain shadows, noise, distortions, and other imperfections as a result of sampling, transmission, improper lighting, or disturbances in the environment during image acquisition. Faulty equipment or their incorrect use also produces distorted poor-quality image. The image enhancement process is used to obtain a second binary or gray-scale image of much improved quality.

Distortions in the image may be *dimensional distortions* due to imperfections in the camera lens or *brightness distortions* due to improper illumination. Overlapping frames or incomplete neutralization of photodetectors in camera produces ghosting. An image may pickup *noise* during transmission from camera to computer. A moving object may produce *blurring* of the image. Finally, the camera might be poorly focused. In all these cases, image enhancement is necessary before processing it to extract the correct information from the image. Several techniques are available to improve the quality of image such as segmentation, smoothing and so on.

9.11.2 Segmentation

A scene may contain one or more objects, which must be distinguished from the background. *Segmentation* is the process of identifying groups of related pixels for locating connected regions or areas of image having similar characteristics. This helps in identifying edges or distinct entities representing possible objects and separating them from the background. Segmentation process divides the image into constituent parts. There are many techniques to segment an image. Three important techniques are discussed here.

(a) *Gray-Level Histogram* The simplest method to segment a gray-scale image into background and foreground (object) area is to convert each pixel to a binary value, representing either background or object. In a digitized image, it is possible to measure the frequency with which each of the quantized levels (grayscale) occurs.

The plot of this frequency for pixels of each gray level g is known as gray-level histogram. For a particular image, with a light object on a dark background, the histogram may look like one shown in Fig. 9.23.

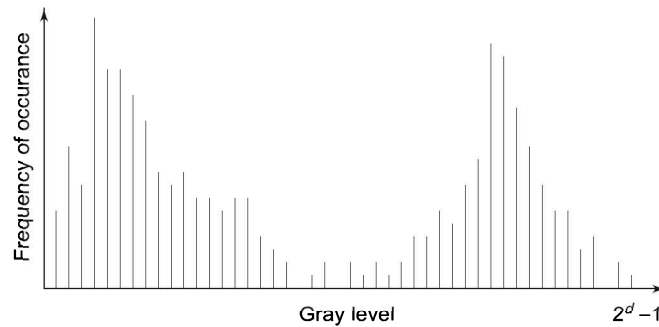


Fig. 9.23 Gray-scale histogram for 2^d gray levels

Each image generates a unique gray-level histogram but it is not possible to work backwards and get the image from the histogram. The histogram in Fig. 9.23 has two peaks. Clearly, the first peak is the background and the second peak represents the object or vice versa. The few scattered stray pixels represent the noise. This method may not work if there are several objects in the image or objects have holes. Sophisticated enhancement techniques may be applied to the histogram. For example, *equalization* may be applied to redistribute gray levels of pixels having low probability of use, which increases the contrast. Other information such as lack of contrast, image saturation and so on, can be easily obtained by examining the minimum and maximum gray levels found in the histogram. The conclusions drawn from the histogram can be employed in robotic vision control systems to control the lighting, camera position and so on, automatically.

(b) *Region Growing* This segmentation technique is based on grouping of pixels having similar attributes into regions. This is also called *pixel aggregation*. The gray-scale raw image is scanned and the region is grown by appending or connecting together the neighbouring pixel that have same property say gray-level, texture, colour and so on. Often the gray-level, that is, the intensity, is the image attribute of interest. Each region is labelled with a unique integer number. A region growing and labelling algorithm is given below assuming gray-level as the property and any pixel belonging to background (gray level = 0) is skipped.

Algorithm 9.1 > Region Growing and Labeling Algorithm

- Step 1** Select any pixel as “seed” (a non-background pixel), note it’s gray-level and assign it a label 1.
- Step 2** Evaluate each unlabelled nonbackground pixel in neighbourhood of the seed pixel. Assign same label to all the neighbouring pixels having same gray level.
- Step 3** Select one of the neighbouring pixels with the same label and call it the seed, go to step 2. If none of the neighbours of all the pixels in the region has same gray level go to step 4. If no unlabelled pixel is found, go to step 6.
- Step 4** Select any unlabelled nonbackground pixel with a gray-level as seed and assign it the next label (“two”, “three” etc.). If no such pixel is found go to step 6.
- Step 5** Go to step 2.
- Step 6** The image scan is over and the image is segmented into regions identified by the pixels of same label.

The region growing assigns a unique identifying label to each region in the image. As an example, consider the 16×16 binary image shown in Fig. 9.24(a) where 0 and 1 represent background and foreground, respectively. The segmented image using the region-growing algorithm is shown in Fig. 9.24(b). Clearly, three different regions, labelled as 1, 2, and 3, are identified. In a nonbinary gray-level image ($d > 1$), regions of different gray-levels may adjoin each other. The adjoining regions may either be merged to get larger regions, if they differ marginally in the intensity values, or left as isolated pixel group. The recognition system can proceed with the segmented information.

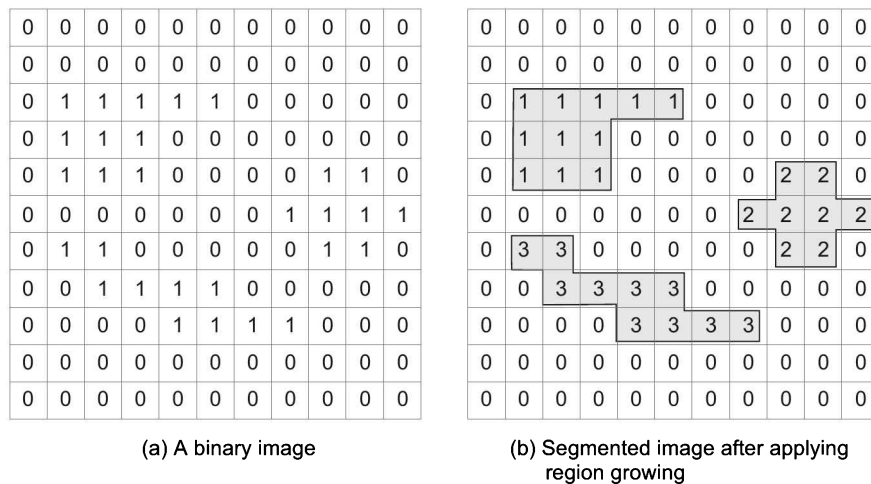


Fig. 9.24 Region growing for segmentation to identify objects in image

(c) *Edge Detection* Another effective technique for segmentation is edge detection. The outline boundary of an object within an image is equivalent to identifying the edges of the object that separate the object from its background.

Algorithms to identify whether an image pixel lies on the edges of an object or not, are known as edge detection algorithms. A common method is based on the intensity change or intensity discontinuity that occurs in adjacent pixels at the boundary or edge of an object. The idea underlying most of the edge detection algorithms is the computation of local gradient of image intensity. The gradient is zero in all regions of constant intensity and is nonzero wherever intensity changes. A high local intensity gradient, indicating a sudden intensity transition, is symptomatic of high probability of presence of a boundary or edge. Thus, the magnitude of the first derivative of intensity function can be used to detect edges in the image.

For binary images, the edge detection can be done by a simple edge following procedure. In a binary image, the change in intensity from 0 to 1 indicates the presence of edge. The image is scanned in a “left-right, top-down scan” until an object pixel (gray level = 1) is found. From there on, the edge is followed by examining the neighbours of the pixels.

To implement this edge following procedure for binary images, a formal concept of defining neighbours of a pixel p is required. There are several methods of defining neighbourhood. For instance, the two horizontal and two vertical neighbours of a pixels p at (x, y) in Fig. 9.25 are given by coordinates

$$(x-1, y), (x+1, y), (x, y-1), (x, y+1)$$

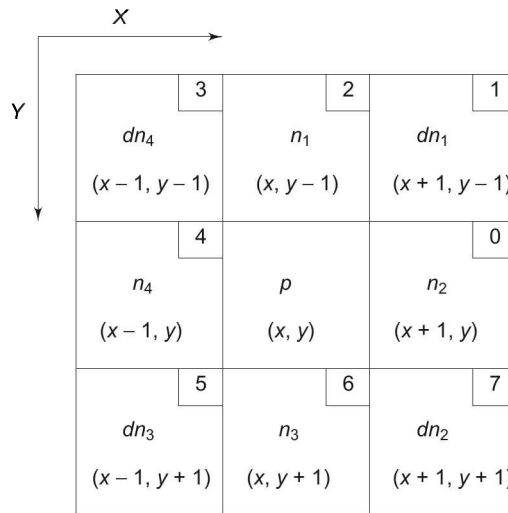


Fig. 9.25 Arrangement of neighbours of a pixel $p(x, y)$

This set of pixels is called *4-neighbours* and the four members of the 4-neighbours set are labelled as n_1 , n_2 , n_3 , and n_4 . In addition, there are four diagonal neighbours of p , labelled as dn_1 , dn_2 , dn_3 , and dn_4 , and their coordinates are:

$$(x-1, y+1), (x+1, y+1), (x+1, y-1), (x-1, y-1)$$

These, together with the 4-neighbours constitute the 8-neighbours of p . These neighbours are illustrated in Fig. 9.25. The numbers in the top right hand corner of each cell indicates the direction of transversal.

The edge-detection procedure for binary images with 4-neighbours is given in Algorithm 9.2.

Algorithm 9.2 > Edge-detection Algorithm for Binary Images

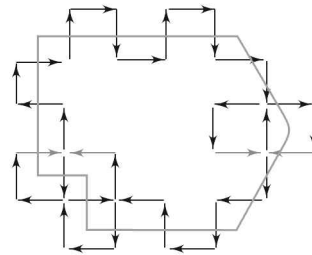
The pixel intensities are 0 and 1 for background and object, respectively.

- Step 1** Do a row-by-row (left to right and top to bottom) scan of the image, starting from origin until an object pixel (value = 1) is found. Record this location as 'start'.
- Step 2** Turn 90° in counterclockwise direction and step into the neighbour.
- Step 3** If the new pixel is same as 'start' go to step 7.
- Step 4** If new pixel is an object pixel, go to step 2.
- Step 5** If new pixel is a background pixel (value = 0), turn 90° clockwise and step into the neighbour.
- Step 6** If the new pixel is same as 'start' go to step 7 else, go to step 4.
- Step 7** Stop when the 'start' pixel in step 1 is reached a second time. Edge detection of one object is complete.

As an example, the Algorithm 9.2 is applied to the binary image in Fig. 9.26(a) and the resulting object boundary is shown in Fig. 9.26(b). The pixels traversed according to the algorithm are indicated with arrows in Fig. 9.26(a).

	1	2	3	4	5	6	7	8
1	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0
3	0	0	1 s	1	1	1	0	0
4	0	0	1	1	1	1	1	0
5	0	0	1	1	1	1	0	0
6	0	0	0	1	1	1	0	0
7	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0

(a)



(b)

Fig. 9.26 Edge detection for a binary image

In case of gray-level images, the local gradient of intensity function has to be computed and to decide whether it is an edge point or not, the familiar procedure of thresholding is applied. The local gradient function $G(x, y)$ is computed for each pixel and the $n \times n$ array of gradient values is obtained. On this array, the algorithm 9.2 is applied with minor modifications. Instead of gray level 0 and 1, the gradient function is used to detect edges. For the local gradient function $G(x, y)$, an 'edge threshold' value G^T is used to detect the edge points as:

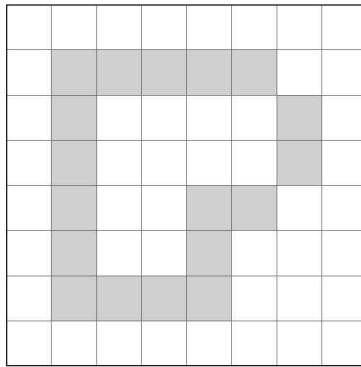
$G(x, y) \geq G^T \rightarrow$ pixel (x, y) is an edge point

$G(x, y) < G^T \rightarrow$ pixel (x, y) is not an edge point

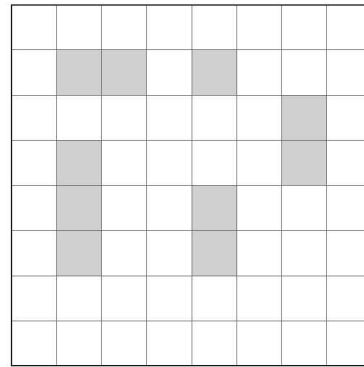
The choice of G^T affects the edge detection and an ideal choice is difficult. The edge detection for a gray level image using local gradient function is illustrated in Fig. 9.27. The local gradient function array with gradient scale between 0 and 10 is shown in Fig. 9.27(a). The edges detected with two different values of threshold values $G^T = 6$ and $G^T = 8$ are shown in Fig. 9.27(b) and (c).

0	1	1	2	3	5	4	1
1	10	9	7	8	7	5	3
0	7	5	4	4	3	8	0
4	10	3	4	5	0	9	4
3	10	1	1	10	7	4	1
4	9	5	2	8	4	2	2
2	7	8	7	6	3	0	2
1	1	4	3	3	2	5	4

(b) Array of local gradient function ($0 \leq G \leq 10$)



(b) $G^T = 6$



(c) $G^T = 8$

Fig. 9.27 Edge detection in gray level image with two different threshold values of local gradient function $G(x, y)$

9.11.3 Smoothing

Smoothing operations are iterative in nature and are used to improve quality of the image. The image intensity is modified using local techniques based on the assumption that the pixel value for a pixel in some sense is similar to that of its neighbours. There are several techniques for smoothing images.

One such technique is neighbourhood averaging. This simple technique generates a smooth image $I'(x, y)$, from image $I(x, y)$, such that the intensity at

every point (x, y) is obtained by averaging the intensity values of pixels of $I(x, y)$ in the predefined neighbourhood of (x, y) . The neighbourhood can be chosen as any suitably sized square or rectangular array of pixels around (x, y) . The degree of smoothing is proportional to the size of neighbourhood used. A major disadvantage of smoothing is that it blurs the edges and other sharp details.

9.11.4 Object Descriptors

An object can be recognized by its characteristics, like geometric features (shape, size, area, perimeter etc.) and other features, for example, number of line segments, corners, arcs, or holes. To respectively identify a given object in different images such characteristics are computed from the image. The extracted geometric features of an image are known as object descriptors. Obviously, any single feature, say perimeter, cannot be a good unique descriptor to identify an object. Therefore, several descriptors are simultaneously obtained for a given object and then, the object may be identified with good statistical confidence level. The image processing for computation of the features from the image data is discussed now. Consider a 2-D binary image shown in Fig 9.28.

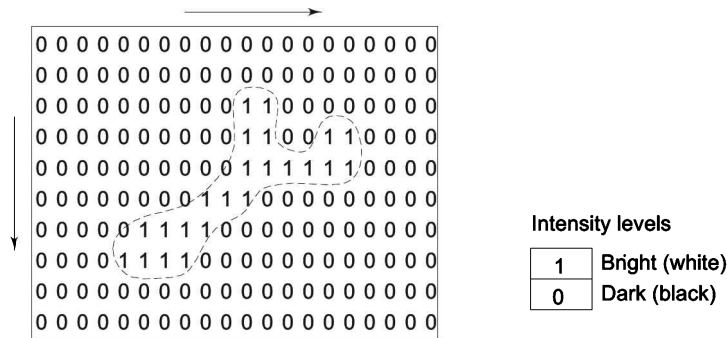


Fig. 9.28 A 2-D binary image array (visual field)

The pixels having a value of '1' represent the object surface and remaining pixels having a value '0' represent the background.

The image intensity function for a binary image of an object O is:

$$I(x, y) = \begin{cases} 1 & \text{for all } (x, y) \in O \text{ (object)} \\ 0 & \text{for all } (x, y) \notin O \text{ (background)} \end{cases} \quad (9.4)$$

This functional definition assumes uniform brightness of object everywhere and background is clearly distinguished. The binary image intensity function for image in Fig. 9.28 is shown in Fig. 9.29. In the view field of $m \times n$ pixels, the object image lies within a rectangular window with corners (a_1, b_1) and (a_2, b_2) as shown in Fig. 9.29.

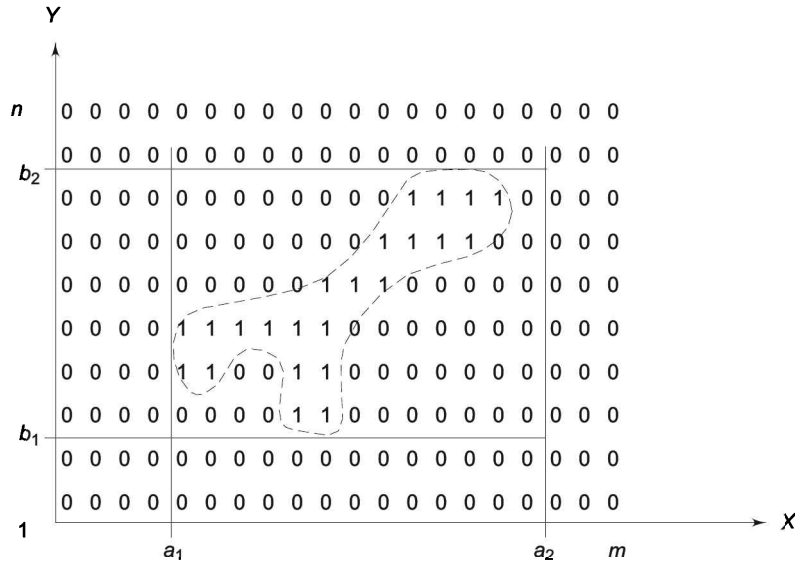


Fig. 9.29 Rectangular window for image in Fig. 9.28

The intensity function in Eq. (9.4) is the mathematical model of the visual field in the form of a $m \times n$ pixel matrix of 0's and 1's. This can be used to extract useful information about the image. Image area, center of gravity, orientation and many other attributes can be computed from the image intensity function. The calculation of these image characteristics is discussed next.

(a) *Image Area* Area, size, or largeness, A of the image is obtained by integrating the function $I(x, y)$ over the view field. Because the image function is a discrete function, integration can be replaced by summation. The area of object O is, thus, given by

$$A = \sum_{x=1}^m \sum_{y=1}^n I(x, y) \quad (9.5)$$

However, computations will be drastically reduced if the summation is carried out over the image portion of visual field, that is, the window between (a_1, b_1) and (a_2, b_2) as shown in Fig. 9.29. Summation over the entire view field of $m \times n$ pixels is considered here for simplicity. Note that the area of binary picture is the sum of all 'ones' in the $m \times n$ pixel matrix.

(b) *Centre of Gravity* The centre of gravity or centroid (x_c, y_c) of the image is

$$x_c = \frac{1}{A} \sum_{x=1}^m \sum_{y=1}^n x I(x, y) \quad (9.6)$$

and

$$y_c = \frac{1}{A} \sum_{x=1}^m \sum_{y=1}^n y I(x, y) \quad (9.7)$$

The area and centre of gravity information is used to identify the position of the object.

(c) *Moments* A sequence of numbers characterizing the shape of the object O is called *moments* of O . The moments M_{jk} are defined as the sums of products of integer powers of row and column numbers of pixels in the object region,

$$M_{jk} \triangleq \sum_{x,y \in O} x^j y^k I(x, y) \quad j \geq 0, k \geq 0 \quad (9.8)$$

The sum of the powers, $(j + k)$ is called the *order of moment* M_{jk} . It follows from Eq. (9.5) and Eq. (9.8) that the zeroeth order moment, M_{00} is equal to the area A , that is

$$M_{00} = A \quad (9.9)$$

Similarly, the center of gravity (x_c, y_c) is given by the first-order moments. From Eq. (9.8), the first moments are

$$M_{10} = \sum_{x=1}^m \sum_{y=1}^n x I(x, y) \quad (9.10)$$

$$\text{and} \quad M_{01} = \sum_{x=1}^m \sum_{y=1}^n y I(x, y) \quad (9.11)$$

Because the size A is given by M_{00} , Eq. (9.9) the normalized first moments provide (x_c, y_c) as

$$x_c = \frac{M_{10}}{M_{00}} \quad (9.12)$$

$$y_c = \frac{M_{01}}{M_{00}} \quad (9.13)$$

Once the centroid of the image is known, moments can also be defined about the center of gravity (x_c, y_c) . These are called central moments and are defined as:

$$\mu_{jk} = \sum_{x,y \in O} (x - x_c)^j (y - y_c)^k I(x, y) \quad j \geq 0, k \geq 0 \quad (9.14)$$

The central moments are equal to moments if centroid is located at the origin. The central moments are invariant to translation of image. Because the object is balanced about the centre of gravity, the first-order central moments are always zero, that is

$$\mu_{10} = \sum_{x=1}^m \sum_{y=1}^n (x - x_c) I(x, y) = 0 \quad (9.15)$$

$$\mu_{01} = \sum_{x=1}^m \sum_{y=1}^n (y - y_c) I(x, y) = 0 \quad (9.16)$$

The second-order central moments of a given image are computed as follows:

$$\mu_{20} = \sum_{x=1}^m \sum_{y=1}^n (x - x_c)^2 I(x, y) \quad (9.17)$$

$$\mu_{02} = \sum_{x=1}^m \sum_{y=1}^n (y - y_c)^2 I(x, y) \quad (9.18)$$

$$\mu_{11} = \sum_{x=1}^m \sum_{y=1}^n (x - x_c)(y - y_c) I(x, y) \quad (9.19)$$

The second-order central moments also give important information about the image. In particular, the moments μ_{02} and μ_{20} represent the moments of inertia of image corresponding to x - and y -axes through the centroid.

(d) *Orientation* The orientation or inclination of the image can also be computed from the second-order central moments. Consider an arbitrary direction represented by a line L passing through centroid (x_c, y_c) of object and making an angle β with the x -axis, as shown in Fig. 9.30. The moment of inertia of image about line L will depend on the angle β . The angle at which this moment is minimized is defined as the inclination of the binary image with respect to x -axis, that is, angle β . The angle β is given by

$$\beta \cong \frac{1}{2} \text{Atan2} (2\mu_{11}, \mu_{20} - \mu_{02}) \quad (9.20)$$

The angle β defines the orientation of the object.

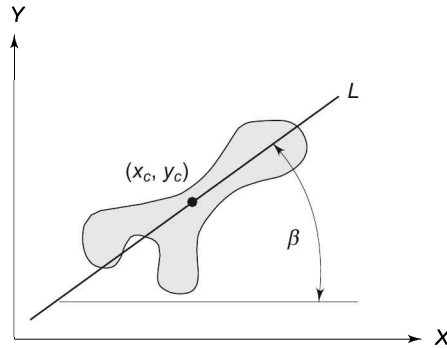


Fig. 9.30 Orientation of object in Fig. 9.29

(e) *Other Attributes* Many other attributes of image can be similarly obtained. Some of these are defined here.

$$\text{Eccentricity} = \frac{\text{Maximum chord length}}{\text{Minimum chord length}} \quad (9.21)$$

The maximum chord length is along the principal axis or major axis of object and minimum chord length is perpendicular to major axis.

The maximum chord length or major diameter D of object O is defined as

$$D = \max_{i,j} \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \quad (9.22)$$

where $p_i = (x_i, y_i)$ and $p_j = (x_j, y_j)$ are pixels in the boundary of O .

$$\text{Thinness} = \frac{(\text{Perimeter})^2}{A} \quad (9.23)$$

$$\text{Roundness} = \frac{(x_c^2 + y_c^2)}{A} \quad (9.24)$$

$$\text{Aspect ratio} = \frac{\text{Length of rectangle enclosing object}}{\text{Width of rectangle enclosing object}} \quad (9.25)$$

For the image in Fig. 9.29,

$$\text{Aspect ratio} = \frac{a_2 - a_1}{b_2 - b_1} \quad (9.26)$$

This may be used to find the rectangle, which gives minimum aspect ratio.

These definitions are not unique and alternative ways of finding these and other features are available in literature.

9.11.5 Object Recognition

Identification of objects contained in the scene is an important task for image processing. Using the features extracted, as defined in the Section 9.11.4, it is possible to find useful information about the object but it is difficult to say what the object is.

The object recognition deals with unique identification of each object in the image. For a robot to be capable to handle parts and their classification, object recognition is required. For robotic application, the algorithms for object recognition should not only be powerful to uniquely identify the object but should be fast enough to match the work cycle.

Image comparison technique is a simple approach for object identification. Images of known objects are stored in the computer and the regions identified in the image are compared with these to recognize the parts in the image. This method is known as *template matching*. Template-matching technique is a subset of more general and vast *pattern recognition* techniques.

Once the image has been improved and segmented, it is compared against the template. In practice, it is rare for an image to match the template exactly. The recognition of the object is probabilistic. The closer the image matches with the template, more probable it is that the object is in fact the object represented by the template. Different templates can be stored for object recognition. A template can be the full image of the standard object, which can be swelled or shrunk to fit the image size, or it can be a set of features of the object in the image such as its area, diameter, aspect ratio and so on. Sufficient number of features must be stored to minimize errors in recognition process. The model template may also be obtained and stored during the vision system training.

7.1 INTRODUCTION TO MACHINE VISION

Machine vision is concerned with the sensing of vision data and its interpretation by a computer. The typical vision system consists of the camera and digitizing hardware, a digital computer, and hardware and software necessary to interface them. This interface hardware and software is often referred to as a preprocessor. The operation of the vision system consists of three functions:

1. Sensing and digitizing image data
2. Image processing and analysis
3. Application

The relationships between the three functions are illustrated in the diagram of Fig. 7.1.

The sensing and digitizing functions involve the input of vision data by means of a camera focused on the scene of interest. Special lighting techniques are frequently used to obtain an image of sufficient contrast for later processing. The image viewed by the camera is typically digitized and stored in computer memory. The digital image is called a *frame of vision data*, and is frequently captured by a hardware device called a *frame grabber*. These devices are capable of digitizing images at the rate of 30 frames per second. The frames consist of a matrix of data representing projections of the scene sensed by the camera. The elements of the matrix are called *picture elements*, or *pixels*. The number of pixels are determined by a sampling process performed on each image frame. A single pixel is the projection of a small portion of the scene which reduces that portion to a single value. The value is measure of the light intensity for that element of the scene. Each pixel intensity is converted into a digital value. (We are ignoring the additional complexities involved in the operation of a color video camera.) We will examine the details of machine vision sensing in Sec. 7.2.

The digitized image matrix for each frame is stored and then subjected to image processing and analysis functions for data reduction and interpretation of the image. These steps are required in order to permit the real-time application of vision analysis required in robotic applications. Typically, an image frame will be thresholded to produce a binary image, and then various feature measurements will further reduce the data representation of the image. This data reduction can change the representation of a frame from several hundred thousand bytes of raw image data to several hundred bytes of feature value data. The resultant feature data can be analyzed in the available time for action by the robot system.

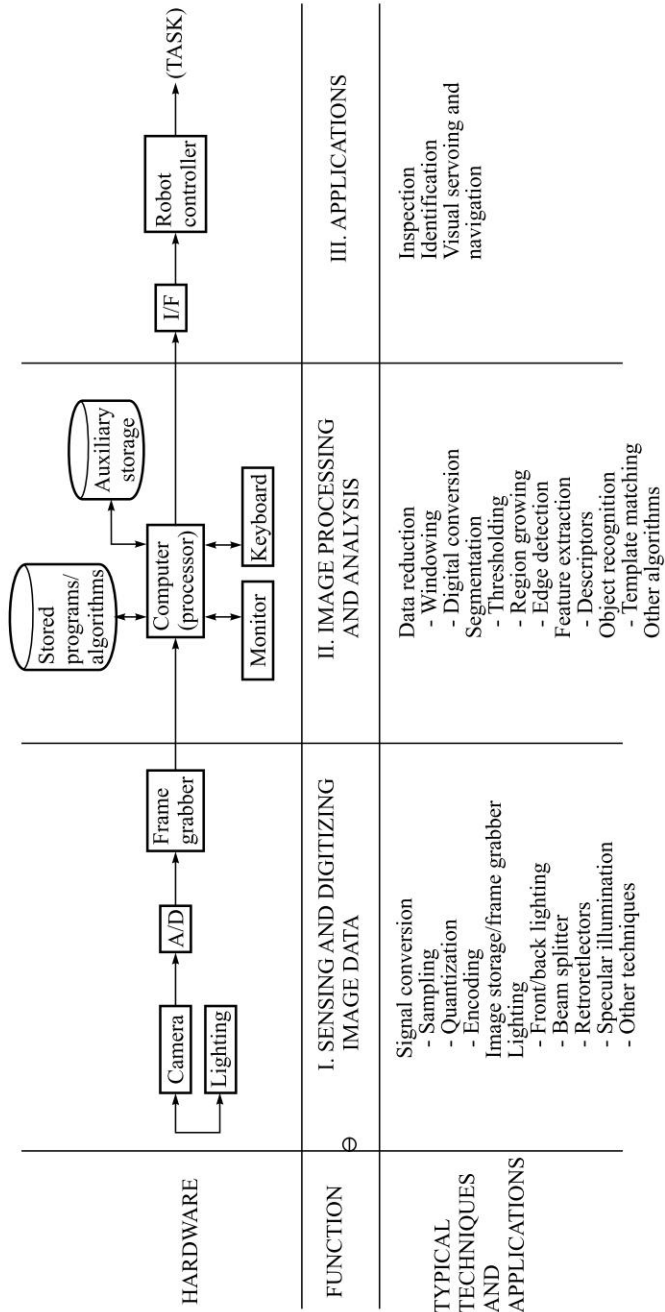


Fig. 7.1 Functions of a machine vision system.

Various techniques to compute the feature values can be programmed into the computer to obtain feature descriptors of the image which are matched against previously computed values stored in the computer. These descriptors include shape

and size characteristics that can be readily calculated from the thresholded image matrix. Some of the important techniques used in the vision system image processing and analysis will be discussed in Sec. 7.3.

To accomplish image processing and analysis, the vision system frequently must be trained. In training, information is obtained on prototype objects and stored as computer models. The information gathered during training consists of features such as the area of the object, its perimeter length, major and minor diameters, and similar features. During subsequent operation of the system, feature values computed on unknown objects viewed by the camera are compared with the computer models to determine if a match has occurred, Sec. 7.4 will discuss training of a vision system.

The third function of a machine vision system is the applications function.

The current applications of machine vision in robotics include inspection, part identification, location, and orientation. Research is ongoing in advanced applications of machine vision for use in complex inspection, guidance, and navigation. In Sec. 7.5, these applications will be discussed.

Vision systems can be classified in a number of ways. One obvious classification is whether the system deals with a two-dimensional or three-dimensional model of the scene. Some vision applications require only a two-dimensional analysis. Examples of two-dimensional vision problems include checking the dimensions of a part or verifying the presence of components on a subassembly.

Many two-dimensional vision systems can operate on a binary image which is the result of a simple thresholding technique. This is based on an assumed high contrast between the object (s) and the background. The desired contrast can often be accomplished by using a controlled lighting system.

Three-dimensional vision systems may require special lighting techniques and more sophisticated image processing algorithms to analyze the image. Some systems require two cameras in order to achieve a stereoscopic view of the scene, while other three-dimensional systems rely on the use of structured light and optical triangulation techniques with a single camera. An example of a structured light system is one that projects a controlled band of light across the object. The light band is distorted according to the three-dimensional shape of the object. The vision system sees the distorted band and utilizes triangulation to deduce the shape.

Another way of classifying vision systems is according to the number of gray levels (light intensity levels) used to characterize the image. In a binary image the gray level values are divided into either of two categories, black or white. Other systems permit the classification of each pixel's gray level into various levels, the range of which is called a gray scale.

7.2 THE SENSING AND DIGITIZING FUNCTION IN MACHINE VISION

Our description of the typical machine vision system in the preceding section identified three functions: sensing and digitizing, image processing and analysis, and application. This and the following sections in the chapter will elaborate on these functions. The present section is concerned with the sensing and digitizing aspects of machine vision.

If we had a voltage signal which, for example, varied over a ± 5 V range, we could then assign the first bit to indicate polarity and use the succeeding 7 bits to identify specific voltage ranges. The number of quantization levels would be $2^7 = 128$ and the quantization level spacing would be 5 V divided by 128 or 0.039 V.

7.2.4 Image Storage

Following A/D conversion, the image is stored in computer memory, typically called a frame buffer. This buffer may be part of the frame grabber or in the computer itself. Various techniques have been developed to acquire and access digital images. Ideally, one would want to acquire a single frame of data in real time. The frame grabber is one example of a video data acquisition device that will store a digitized picture and acquire it in $= \frac{1}{30}$ s. Digital frames are typically quantized to 8 bits per pixel. However, a 6 bit buffer is adequate since the average camera system cannot produce 8 bits of noise-free data. Thus the lower-order bits are dropped as a means of noise cleaning. In addition, the human eye can only resolve about $2^6 = 64$ gray levels. A combination of row and column counters are used in the frame grabber which are synchronized with the scanning of the electron beam in the camera. Thus, each position on the screen can be uniquely addressed. To read the information stored in the frame buffer, the data is 'grabbed' via a signal sent from the computer to the address corresponding to a row-column combination. Such frame grabber techniques have become extremely popular and are used frequently in vision systems.

7.3 IMAGE PROCESSING AND ANALYSIS

The discussion in the preceding section described how images are obtained, digitized, and stored in a computer. For use of the stored image in industrial applications, the computer must be programmed to operate on the digitally stored image. This is a substantial task considering the large amount of data that must be analyzed. Consider an industrial vision system having a pixel density of 350 pixels per line and 280 lines (a total of 98,000 picture elements), and a 6-bit register for each picture element to represent various gray levels; this would require a total of $98,000 \times 6 = 588,000$ bits of data for each $\frac{1}{30}$ s. This is a formidable amount of data to be processed in a short period of time and has led to various techniques to reduce the magnitude of the imageprocessing problem. These techniques include:

1. Image data reduction
2. Segmentation
3. Feature extraction
4. Object recognition

These techniques of image data analysis are discussed in the following subsections.

7.3.1 Image Data Reduction

In image data reduction, the objective is to reduce the volume of data. As a preliminary step in the data analysis, the following two schemes have found common usage for data reduction:

1. Digital conversion
2. Windowing

The function of both schemes is to eliminate the bottleneck that can occur from the large volume of data in image processing.

Digital conversion reduces the number of gray levels used by the machine vision system. For example, an 8-bit register used for each pixel would have $2^8 = 256$ gray levels. Depending on the requirements of the application, digital conversion can be used to reduce the number of gray levels by using fewer bits to represent the pixel light intensity. Four bits would reduce the number of gray levels to 16. This kind of conversion would significantly reduce the magnitude of the image-processing problem.

Example 7.4

For an image digitized at 128 points per line and 128 lines, determine (a) the total number of bits to represent the gray level values required if an 8-bit A/D converter is used to indicate various shades of gray, and (b) the reduction in data volume if only black and white values are digitized.

- (a) For gray scale imaging with $2^8 = 256$ levels of gray

Number of hits = $128 \times 128 \times 256 = 4,194,304$ bits.

- (b) For black and white (binary bit conversion)

Number of bits = $128 \times 128 \times 2 = 16,384$ bits

Reduction in data volume = $4,194,304 - 16,384$

= 4,177,920 bits.

Windowing involves using only a portion of the total image stored in the frame buffer for image processing and analysis. This portion is called the *window*. For example, for inspection of printed circuit boards, one may wish to inspect and analyze only one component on the board. A rectangular window is selected to surround the component of interest and only pixels within the window are analyzed. The rationale for windowing is that proper recognition of an object requires only certain portions of the total scene.

7.3.2 Segmentation

Segmentation is a general term which applies to various methods of data reduction. In segmentation, the objective is to group areas of an image having similar characteristics or features into distinct entities representing parts of the image. For example, boundaries (edges) or regions (areas) represent two natural segments of an image. There are many ways to segment an image.

Three important techniques that we will discuss are:

1. Thresholding
2. Region growing
3. Edge detection

In its simplest form, *thresholding* is a binary conversion technique in which each pixel is converted into a binary value, either black or white. This is accomplished by utilizing a frequency histogram of the image and establishing what intensity

(gray level) is to be the border between black and white. This is illustrated for an image of an object as shown in Fig. 7.6. Fig. 7.6(a) shows a regular image with each pixel having a specific gray tone out of 256 possible gray levels. The histogram of Fig. 7.6(b) plots the frequency (number of pixels) versus the gray level for the image. For histograms that are bimodal in shape, each peak of the histogram represents either the object itself or the background upon which the object rests. Since we are trying to differentiate between the object and background, the procedure is to establish a threshold (typically between the two peaks) and assign, for example, a binary bit 1 for the object and 0 for the background. The outcome of this thresholding technique is illustrated in the binary-digitized image of Fig. 7.6(c). To improve the ability to differentiate, special lighting techniques must often be applied to generate a high contrast.

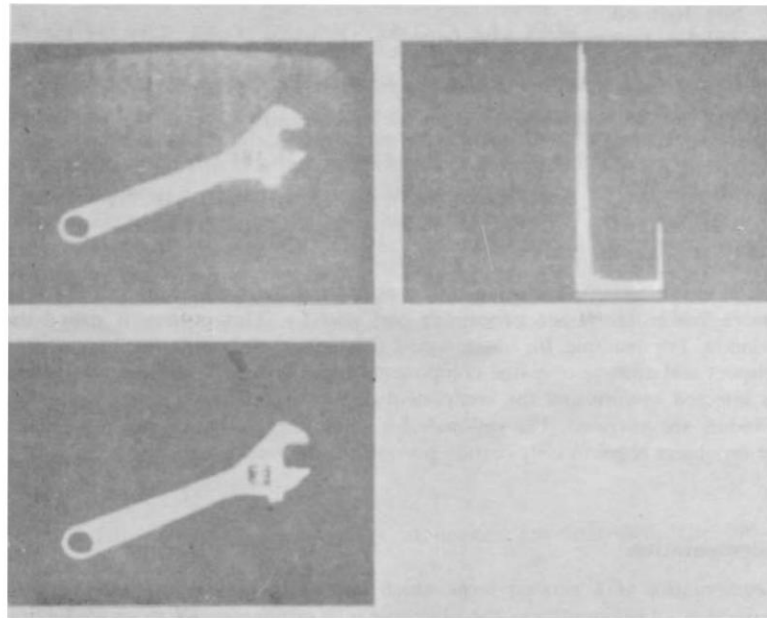


Fig. 7.6 Obtaining a binary image by thresholding: (a) Image of object with all gray-levels present, (b) Histogram of image, (c) Binary image of object after thresholding. (Photos courtesy: Robotics Laboratory, Lehigh University)

It should be pointed out that the above method of using a histogram to determine a threshold is only one of a large number of ways to threshold an image. It is however the method used by many of the commercially available robot vision systems today. Such a method is said to use a global threshold for the entire image. In some cases, this is not possible and a local thresholding method as described below may be employed.

When it is not possible to find a single threshold for an entire image (for example, if many different objects occupy the same scene, each having different levels of intensity), one approach is to partition the total image into smaller rectangular areas and determine the threshold for each window being analyzed.

Thresholding is the most widely used technique for segmentation in industrial vision applications. The reasons are that it is fast and easily implemented and that the lighting is usually controllable in an industrial setting.

Once thresholding is established for a particular image, the next step is to identify particular areas associated with objects within the image. Such regions usually possess uniform pixel properties computed over the area. The pixel properties may be multidimensional; that is, there may be more than a single attribute that can be used to characterize the pixel (e.g., color and light intensity). We will avoid this complication and confine our discussion to single pixel attributes (light intensity) of a region.

Region growing is a collection of segmentation techniques in which pixels are grouped in regions called *grid elements* based on attribute similarities. Defined regions can then be examined as to whether they are independent or can be merged to other regions by means of an analysis of the difference in their average properties and spatial connectiveness. For instance, consider an image as depicted in Fig. 7.7(a). To differentiate between the objects and the background, assign 1 for any grid element occupied by an object and 0 for background elements. It is common practice to use a square sampling grid with pixels spaced equally along each side of the grid. For the two-dimensional image of a key as shown, this would give the pattern indicated in Fig. 7.7(b). This technique of creating 'runs' of 1s and 0s is often used as a first-pass analysis to partition the image into identifiable segments or 'blobs.' Note that this simple procedure did not identify the hole in the key of Fig. 7.7(a). This could be resolved by decreasing the distance between grid points and increasing the accuracy with which the original image is represented.

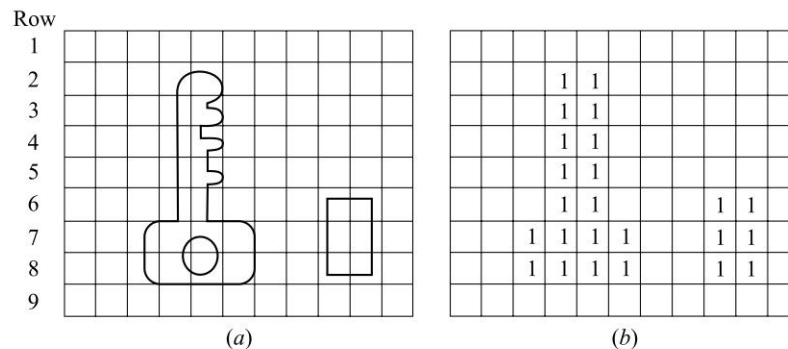


Fig. 7.7 Image segmentation: (a) Image pattern with grid, (b) Segmented image after runs test.

For a simple image such as a dark blob on a light background, a runs technique can provide useful information. For more complex images, this technique may not provide an adequate partition of an image into a set of meaningful regions. Such regions might contain pixels that are connected to each other and have similar attributes, for example, gray level. A typical region-growing technique for complex images could have the following procedure:

1. Select a pixel that meets a criterion for inclusion in a region. In the simplest case, this could mean select white pixel and assign a value of 1.

2. Compare the pixel selected with all adjacent pixels. Assign an equivalent value to adjacent pixels if an attribute match occurs.
3. Go to an equivalent adjacent pixel and repeat process until no equivalent pixels can be added to the region.

This simple procedure of 'growing' regions around a pixel would be repeated until no new regions can be added for the image.

The region growing segmentation technique described here is applicable when images are not distinguishable from each other by straight thresholding or edge detection techniques. This sometimes occurs when lighting of the scene cannot be adequately controlled. In industrial robot vision systems, it is common practice to consider only edge detection or simple thresholding. This is due to the fact that lighting can be a controllable factor in an industrial setting and hardware/computational implementation is simpler.

Edge detection considers the intensity change that occurs in the pixels at the boundary or edges of a part. Given that a region of similar attributes has been found but the boundary shape is unknown, the boundary can be determined by a simple edge following procedure. This can be illustrated by the schematic of a binary image as shown in Fig. 7.8. For the binary image, the procedure is to scan the image until a pixel within the region is encountered. For a pixel within the region, turn left and step, otherwise, turn right and step. The procedure is stopped when the boundary is traversed and the path has returned to the starting pixel. The contour-following procedure described can be extended to gray level images.

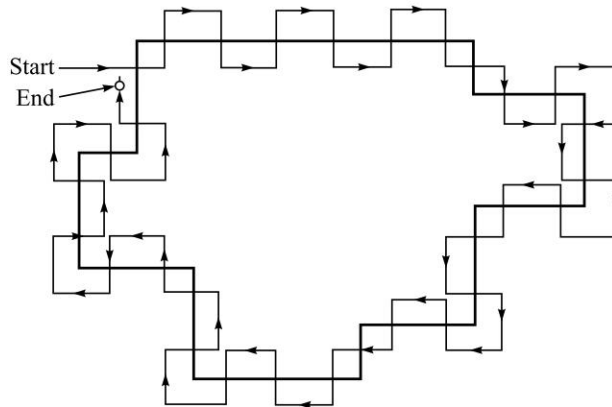


Fig. 7.8 Edge following procedure to detect the edge of a binary image.

7.3.3 Feature Extraction

In machine vision applications, it is often necessary to distinguish one object from another. This is usually accomplished by means of features that uniquely characterize the object. Some features of objects that can be used in machine vision include area, diameter, and perimeter. A feature, in the context of vision systems, is a single parameter that permits ease of comparison and identification. A list of

some of the features commonly used in vision applications is given in Table 7.2. The techniques available to extract feature values for two-dimensional cases can be roughly categorized as those that deal with boundary features and those that deal with area features. The various features can be used to identify the object or part and determine the part location and/or orientation.

The region-growing procedures described before can be used to determine the area of an object's image. The perimeter or boundary that encloses a specific area can be determined by noting the difference in pixel intensity at the boundary and simply counting all the pixels in the segmented region that are adjacent to pixels not in the region, that is, on the other side of the boundary. An important objective in selecting these features is that the features should not depend on position or orientation. The vision system should not be dependent on the object being presented in a known and fixed relationship to the camera.

Table 7.2 Basic features and measures for object identification for two-dimensional objects.

Gray level (maximum, average, or minimum)

Area

Perimeter length

Diameter

Minimum enclosing rectangle

Center of gravity—For all pixels (n) in a region where each pixel is specified by (x , y) coordinates, the x and y coordinates of the center of gravity are defined as

$$C.G._x = \frac{1}{n} \sum_x x$$

$$C.G._y = \frac{1}{n} \sum_y y$$

Eccentricity: A measure of 'elongation' Several measures exist of which the simplest is

$$\text{Eccentricity} = \frac{\text{Maximum chord length } A}{\text{Maximum chord length } B}$$

where maximum chord length B is chosen perpendicular to A .

Aspect ratio—The length-to-width ratio of a boundary rectangle which encloses the object.

One objective is to find the rectangle which gives the minimum aspect ratio.

Thinness—This is a measure of how thin an object is. Two definitions are in use

$$(a) \quad \text{Thinness} = \frac{(\text{Perimeter})^2}{\text{Area}}$$

This is also referred to as compactness.

$$\text{Diameter } (b) \quad \text{Thinness} = \frac{\text{Diameter}}{\text{Area}}$$

The diameter of an object, regardless of its shape, is the maximum distance obtainable for two points on the boundary of an object.

Contd.

Holes—Number of holes in the object.

Moments—Given a region, R , and coordinates of the points (x, y) in or on the boundary of the region, the pq th order moment of the image of the region is given as

$$M_{pq} = \sum_{x, y} x^p y^q$$

The preceding measures provide some basic methods to analyze images in a two-dimensional plane. Various other measures exist for the three-dimensional case as well. To illustrate some of the two-dimensional definitions and measures, the following example is provided:

Example 7.5

Consider the schematic of the image in Fig. 7.9. Determine the area, the minimum aspect ratio, the diameter, the centroid, and the thinness measures of the image.

For the image, the minimum boundary rectangle is as shown. For ease of calculation, the origin is translated to O' with x', y' coordinates. The area may be determined from the moment, $M_{O'O'}$ as

$$M_{O'O'} = \sum_{x', y'} x' y' = 24 \text{ pixels}$$

The minimum aspect ratio would use the minimum boundary rectangle and would simply be

$$\text{Minimum aspect ratio} = \frac{\text{Length}}{\text{Width}} = \frac{9}{4}$$

with the diameter = 9 pixels. The centroid position for the $n = 24$ pixels in the region would be calculated from

$$\text{C.G.}_{x'} = \frac{1}{n} \sum_{x'} x'$$

$$\text{C.G.}_{y'} = \frac{1}{n} \sum_{y'} y'$$

where pixel location is taken at the midpoint of each pixel cell. (C.G. =

$$\text{C.G.}_{x'} = \frac{1}{24} \left[4 \left(\frac{1}{2} \right) + 4 \left(\frac{3}{2} \right) + 4 \left(\frac{5}{2} \right) + 2 \left(\frac{7}{2} \right) + \dots + 2 \left(\frac{17}{2} \right) \right]$$

$$= \frac{1}{24} (90) = \frac{15}{4} \text{ units}$$

$$\text{C.G.}_{y'} = \frac{1}{24} \left[3 \left(\frac{1}{2} \right) + 9 \left(\frac{3}{2} \right) + 9 \left(\frac{5}{2} \right) + 3 \left(\frac{7}{2} \right) \right]$$

$$= \frac{96}{48} = 2 \text{ units}$$

center of gravity.) The centroid is indicated in Fig. 7.9. Of the two thinness measures defined, the calculations would result in

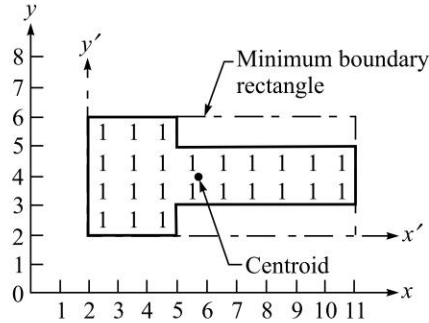


Fig. 7.9 Schematic of pixel pattern for Example 7.5.

$$\text{Compactness} = \frac{(\text{Perimeter})^2}{\text{Area}} = \frac{26^2}{24} = 28.17$$

$$\text{Thinness} = \frac{\text{Diameter}}{\text{Area}} = \frac{9}{24} = \frac{3}{8}$$

7.3.4 Object Recognition

The next step in image data processing is to identify the object the image represents. This identification problem is accomplished using the extracted feature information described in the previous subsection. The recognition algorithm must be powerful enough to uniquely identify the object. Object recognition techniques used in industry today may be classified into two major categories:

1. Template-matching techniques
2. Structural techniques

Template-matching techniques are a subset of the more general statistical pattern. Recognition techniques that serve to classify objects in an image into predetermined categories. The basic problem in template matching is to match the object with a stored pattern feature set defined as a model template. The model template is obtained during the training procedure in which the vision system is programmed for known prototype objects. These techniques are applicable if there is not a requirement for a large number of model templates. The procedure is based on the use of a sufficient number of features to minimize the frequency of errors in the classification process. The features of the object in the image (e.g., its area, diameter, aspect ratio, etc.) are compared to the corresponding stored values. These values constitute the stored template. When a match is found, allowing for certain statistical variations in the comparison process, then the object has been properly classified.

Structural techniques of pattern recognition consider relationships between features or edges of an object. For example, if the image of an object can be

subdivided into four straight lines (the lines are called primitives) connected at their end points, and the connected lines are at right angles, then the object is a rectangle. This kind of technique, known *as* syntactic pattern recognition, is the most widely used structural technique. Structural techniques differ from decision-theoretic techniques in that the latter deals with a pattern on a quantitative basis and ignores for the most part interrelationships among object primitives. A detailed discussion of pattern recognition techniques is the subject of complete books and is beyond the scope of this text.

It can be computationally time consuming for complete pattern recognition. Accordingly, it is often more appropriate to search for simpler regions or edges within an image. These simpler regions can then be used to extract the required features. The majority of commercial robot vision systems make use of this approach to the recognition of two-dimensional objects. The recognition algorithms are used to identify each segmented object in an image and assign it to a classification (e.g., nut, bolt, flange, etc.)

7.4 TRAINING THE VISION SYSTEM

The purpose of vision system training is to program the vision system with known objects. The system stores these objects in the form of extracted feature values which can be subsequently compared against the corresponding feature values from images of unknown objects. Typical features used in machine vision were discussed in the previous section and summarized in Table 7.2.

Training of the vision system should be carried out under conditions as close to operating conditions as possible. Physical parameters such as camera placement, aperture setting, part position, and lighting are the critical conditions that should be simulated as closely as possible during the training session.

Vision system manufacturers have developed application software for each individual system marketed. The software is typically based on a high-level programming language. For example, Object Recognition Systems Inc. uses the 'C' language, and Automatix Inc. uses their internally developed language called RAIL (for Robot Automatic Incorporated Language). There are two versions of RAIL, one for automated vision systems and the other for robot programming. We will present portions of the robot programming language in the appendix to Chap. 9.

7.5 ROBOTIC APPLICATIONS

Many of the current applications of machine vision are inspection tasks that do not involve the use of an industrial robot. A typical application is where the machine vision system is installed on a high-speed production line to accept or reject parts made on the line. Unacceptable parts are ejected from the line by some mechanical device that is communicating with the vision system.

Machine vision applications can be considered to have three levels of difficulty. These levels depend on whether the object to be viewed is controlled in position and/

or appearance. Controlling the position of an object in a manufacturing environment usually requires precise fixturing. Controlling the appearance of an object is accomplished by lighting techniques. Also, appearance is influenced by the object's surface texture or coloration. The three levels of difficulty used to categorize machine vision applications in an industrial setting are:

1. The object can be controlled in both position and appearance.
2. Either position or appearance of the object can be controlled but not both.
3. Neither position nor appearance of the object can be controlled.

The third level of difficulty requires advanced vision capabilities. The objective in engineering the vision application is to lower the level of difficulty involved, thereby reducing the level of sophistication of the vision system required in the application. For example, one problem that occurs in object recognition is that the recognition process is facilitated if the object is in a known position and orientation. Parts in a factory are typically not positioned and oriented in this manner. This problem can be reduced from a third level to a first level of difficulty by fixturing the parts and using techniques such as structured lighting to control the appearance.

In this section, the use of machine vision in robotic applications will be emphasized. Robotic applications of machine vision fall into the three broad categories listed below:

1. Inspection
2. Identification
3. Visual servoing and navigation

The first category is one in which the primary function is the inspection process. This is carried out by the machine vision system, and the robot is used in a secondary to support the application. The objectives of machine vision inspection include checking for gross surface defects, discovery of flaws in labeling (during final inspection of the product package), verification of the presence of components in assembly, measuring for dimensional accuracy, and checking for the presence of holes and other features in a part. When these kinds of inspection operations are performed manually, there is a tendency for human error. Also, the time required in most manual inspection operations requires that the procedures be accomplished on a sampling basis. With machine vision, these procedures are carried out automatically, using 100 percent inspection, and usually in much less time. In Chap. 15, we will return to robotic inspection in which many of the applications of robots make use of machine vision.

The second category, *identification*, is concerned with applications in which the purpose of the machine vision system is to recognize and classify an object rather than to inspect it. Inspection implies that the part must be either accepted or rejected. Identification involves a recognition process in which the 'part itself, or its position and/or orientation, is determined. This is usually followed by a subsequent decision and action taken by the robot. Identification applications of machine vision include part sorting, palletizing and depalletizing, and picking parts that are randomly oriented from a conveyor or bin.

In the third application category, *visual servoing and navigational control*, the purpose of the vision system is to direct the actions of the robot (and other devices in the robot cell) based on its visual input. The generic example of robot visual servoing is where the machine vision system is used to control the trajectory of the robot's end effector toward an object in the workspace. Industrial examples of this application include part positioning, retrieving parts moving along a conveyor, retrieving and reorienting parts moving along a conveyor, assembly, bin picking, and seam tracking in continuous arc welding. The use of machine vision in arc welding is described in Chap. 14 when the application of welding robots will be discussed in detail. An example of navigational control would be in automatic robot path planning and collision avoidance using visual data. Clearly, the visual data are just an important input in this type of task and a great deal of intelligence is required in the controller to use the data for navigation and collision avoidance. This and the visual servoing tasks remain important research topics and are not now viable applications of robot vision systems.

The *bin-picking* application is an interesting and complex application of machine vision in robotics which involves both identification and servoing. Bin picking involves the use of a robot to grasp and retrieve randomly oriented parts out of a bin or similar container. The application is complex because parts will be overlapping each other. The vision system must first recognize a target part and its orientation in the container, and then it must direct the end effector to a position to permit grasping and pickup. The difficulty is in the fact that the target part is jumbled together with many other parts (probably identical parts), and the conditions of contrast between the target and its surroundings are far from ideal for part recognition.

Solution of the bin-picking problem owes much to the pioneering work in vision research at the University of Rhode Island. At the time of writing, there are two commercially available bin-picking systems, one offered by Object Recognition Systems, Inc. (ORS) called the i-bot 1 system, and the other by General Electric Co. called Bin Vision. We will describe the ORS product to illustrate the operation of these systems. Fig. 7.10 shows the i-bot system. It utilizes a Unimation PUMA robot to pick parts from the bin. The system is capable of identifying the position and orientation of three parts in a 5-s image-processing cycle. However, the parts must have a relatively large aspect ratio (length-to-width ratio) for the system to operate effectively. Also, the objects in the bin must be the same in terms of features such as size, shape, and texture.

Seam tracking in continuous arc welding is another example of visual servoing and navigation in robotics vision systems. Because the vision system must operate in the presence of an arc-welding torch, special problems in the interpretation of the vision image must be solved. A typical solution uses a form of structured light to project a known pattern on the parts to be welded and uses the observed geometric distortion to determine the path and other parameters required for a successful welding operation. We will discuss this and other sensor technologies used in robotic arc welding in Chap. 14.



Fig. 7.10 *i-bot robot-vision system. (Photo courtesy: Object Recognition Systems, Inc.)*

In addition to the three main categories of machine vision applications in robotics, there are other special purpose vision applications. An example of these special applications is one, in which the vision system functions as a range finder using triangulation or other techniques to determine the distance to the object.

Machine vision, coupled with the force and torque sensors discussed in Chap. 6, can immensely enhance a robot's applicability in manufacturing. Representative tasks in which such highly sensed systems could be applied include manipulation of parts during deburring, flash removal, and assembly. We will discuss the particular technical problems encountered in robotic assembly and the need for sensors in this application in Chap. 15.

Problems

7.1 Consider a vision system which provides one frame of 256 lines every $\frac{1}{2}$ s.

The system is a raster scan system. Assume that the time for the electron beam to move from one line to the next takes 15 per cent of the time to scan a single line. Determine the sampling rate for the system if it is specified that there will be 320 pixels on each line.

CHAPTER 9

Image Processing and Analysis with Vision Systems

9.1 Introduction

A very large body of work is associated with vision systems, image processing, and pattern recognition that addresses many different hardware and software related topics on these subjects. This information has been accumulated since the 1950s, and with the added interest in the subject from different sectors of the industry and economy, it is rapidly growing. The enormous number of papers published every year indicates that many useful techniques constantly appear in the literature. At the same time, it also means that many of these techniques may be suitable for certain applications only. In this chapter, we will study and discuss some fundamental techniques for image processing and image analysis, with a few examples of routines developed for certain purposes. This chapter does not intend to be a complete survey of all possible vision routines, but only an introduction. If interested, it is recommended that you continue studying the subject through other references.

9.2 Basic Concepts

The following sections include some fundamental definitions of terms and basic concepts that we will use throughout the chapter.

9.2.1 Image Processing versus Image Analysis

Image processing relates to the preparation of an image for later analysis and use. Images, as captured by a camera or other similar techniques (such as a scanner), are not necessarily in a form that can be used by image analysis routines. Some may need improvement to

reduce noise; some may need to be simplified; others may need to be enhanced, altered, segmented, filtered, and so on. Image processing is the collection of routines and techniques that improve, simplify, enhance, and otherwise alter an image.

Image analysis is the collection of processes by which a captured and processed image is analyzed to extract information about the content and to identify objects or other related facts about the objects within the image or the environment.

9.2.2 Two- and Three-Dimensional Image Types

Although all scenes are three-dimensional, images can either be two-dimensional (lacking depth information) or three-dimensional (containing depth information). Most images with which we normally deal, obtained by cameras, are two-dimensional. However, other systems such as Computed Tomography (CT) and CAT-scans create three-dimensional images that contain depth information. Therefore, these images can be rotated about different axes in order to better visualize the depth information. A two-dimensional image is extremely useful for many applications even though it has no depth information. This includes feature extraction, inspection, navigation, parts handling, and many more.

Three-dimensional images are used with applications that require motion detection, depth measurement, remote sensing, relative positioning, and navigation. CAD/CAM-related operations also require three-dimensional image processing, as do many inspection and object recognition applications. For three-dimensional images, either X-rays or ultrasonics are used to get images of one slice of the object at a time; later, all images are put together to create a three-dimensional image representation of the internal characteristics of the object.

All three-dimensional vision systems share the problem of coping with many-to-one mapping of scenes to images. To extract information from these scenes, image processing techniques are combined with artificial intelligence techniques. When the system is working in environments with known characteristics (e.g., controlled lighting), it functions with high accuracy and speed. On the contrary, when the environment is unknown or noisy and uncontrolled (e.g., underwater operations), the systems are not very accurate and require additional processing of the information, and therefore, operate at lower speeds.

9.2.3 The Nature of an Image

An image is a representation of a real scene, either in black and white (B/W) or in color, and either in print or in digital form. Printed images may be reproduced either by multiple colors and gray scales, such as CMYK in color print or halftone black and white print, or by a single ink source. For example, to reproduce a photograph with real halftones, we use multiple gray inks, which when combined, produce a somewhat realistic image. However, in most print applications, only one ink color is available (such as black ink on white paper in a newspaper or copier). In this case, all gray levels must be produced by changing the ratio of black versus white areas (the size of the black dot). Imagine that a picture to be printed is divided into small sections. In each section, if the ink portion of the section is smaller than the white, the section will look a lighter gray

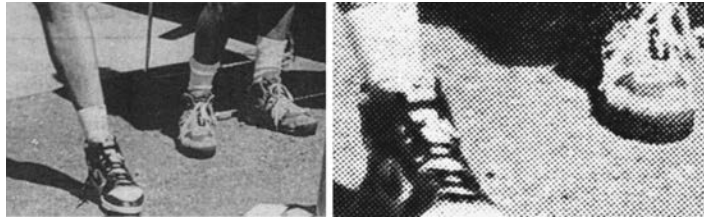


Figure 9.1 Examples of how gray intensities are created in printed images. In print, only one color ink is used, while the ratio of black to white area of the pixel is changed to create different gray levels. (Courtesy of Mustang Daily, Cal Poly.)

(Figure 9.1). If the black ink area is larger than the white area, it will look a darker gray. By changing the size of the printed dot, many gray levels may be produced and, collectively, a gray scale picture may be printed.

Similar to printed images, electronic and digital images are also divided into small sections called picture cells, or *pixels* (in three-dimensional images, it is called volume cell or *voxel*), where the size of all pixels is the same. To capture an image, the intensity of each pixel is measured and recorded; similarly, to recreate an image, the intensity of light at each pixel location is varied. Therefore, an image file is the collection of the data representing the light intensities of a large number of pixels. This file can be recreated, processed, modified, or analyzed. A color image is essentially the same, except that the original image is separated into three images of red, green, and blue before capturing and digitization. When the three colors with different intensities at each pixel location are superimposed, color images are recreated.

9.2.4 Acquisition of Images

There are two types of vision cameras: analog and digital. Analog cameras are no longer common but are still around and used to be the standard camera at television stations. Digital cameras are the current standard and are practically all the same. A digital moving-picture camera is similar to a digital still-camera plus a recording section; otherwise, the mechanism of image acquisition is the same. Whether the captured image is analog or digital, in vision systems, the image is eventually digitized. In digital systems, all data is in binary form and is stored in a computer file or memory device. Therefore, we ultimately deal with a file of numbers 0 and 1, from which we extract information and make decisions.

Appendix B presents a short discussion about analog and digital cameras and how the image is captured. The final outcome of these systems is a file that contains sequential pixel-location and pixel-intensity data that we use in our discussions. Refer to Appendix B for understanding the fundamental issues about image acquisition. For more information about details of these systems, refer to digital data acquisition references.

9.2.5 Digital Images

The light intensities at each pixel location are measured and converted to digital form regardless of the type of camera or image acquisition system. The data is either stored in

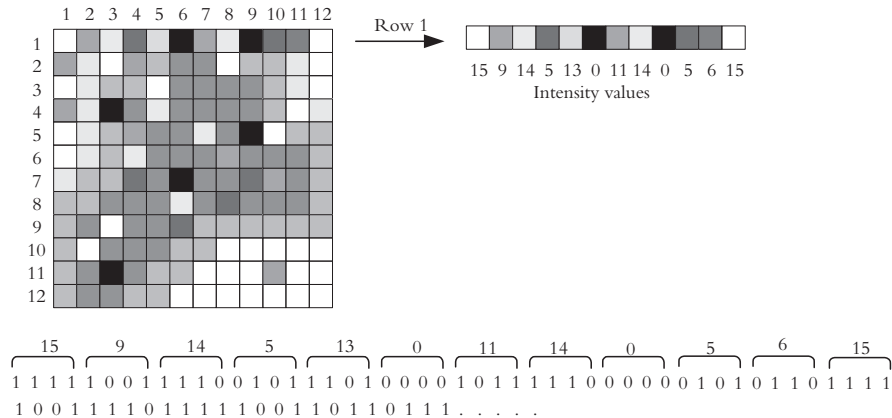


Figure 9.2 An image and the binary representation of its first row using 4 bits per pixel.

memory, in a file, or in recording devices with an image format such as TIFF, JPG, Bitmap, and so on, or is displayed on a monitor. Since it is digitized, the stored information is a collection of 0s and 1s that represent the intensity of light at each pixel; a digitized image is nothing more than a computer file that contains the collection of these 0s and 1s, sequentially stored to represent the intensity of light at each pixel. However, these files can be accessed and read by a program, duplicated and manipulated, or rewritten in a different form. Vision routines generally access this information and perform some function on the data and either display the result or store the manipulated result in a new file. A fundamental issue is to be able to extract information or manipulate this collection of 0 and 1 values in a meaningful way.

To understand this better, consider the simple low resolution image in Figure 9.2. Each pixel is referred to by row and column numbers. Assuming the system is digitized with only 4 bits (we will discuss this further shortly), there will be up to $2^4 = 16$ distinct light intensities possible. The sequence of 0 and 1 numbers representing the first row of the image will look as shown in the figure (all with only 4 bits per pixel). Different file formats list these numbers differently. In a simple Portable Gray Map (PGM) format, the intensities are listed sequentially as shown. A header at the beginning of the file will indicate the number of pixels in each row and column (in this case, it is 12×12). The program knows that every 4 bits is 1 pixel. Therefore, it can access each pixel intensity directly. However, as you notice, the file is reduced to a string of 0 and 1 values, to which, image processing routines are applied, and from which, information is extracted. Now imagine the size of the string of 0 and 1 values that represents a large image (sometimes in mega-pixel range) at up to 24 bits per pixel, for three primary colors.

An image with different gray levels at each pixel location is called a gray image. A color image results by superimposing three images of red, green, and blue hues (RGB), each with a varying intensity and each equivalent to a gray image (but in one of the three hues). Therefore, when the image is digitized, it will similarly have strings of 0s and 1s for each hue (an alternative way is to assign a number to each color, all declared in a header at the beginning of the image file. Then the number representing the pixel represents the color reference and intensity). A binary image is an image where each pixel is either fully light or fully dark, either a 0 or a 1. To achieve a binary image, in most cases, a gray image is

converted using the histogram of the image and a cut-off value called a threshold. A histogram of the pixel gray levels will determine the distribution of the different gray levels. We can pick a value that best determines a cut-off level with the least distortion and use this value as a threshold to assign a 0 (or off) to all pixels whose gray levels are below the threshold value and to assign a 1 (or on) to all pixels whose gray values are above the threshold. Changing the threshold will change the binary image. The advantage of a binary image is that it requires far less memory, and it can be processed much faster than gray or colored images.

9.2.6 Frequency Domain versus Spatial Domain

Many processes used in image processing and analysis are either based on frequency domain or on spatial domain. In frequency domain processing, the frequency spectrum of the image is used to alter, analyze, or process the image. In this case, the individual pixels and their contents are not used. Instead, a frequency representation of the whole image is used for the process. In spatial domain processing, the process is applied to the individual pixels of the image. As a result, each pixel is affected directly by the process. Both techniques are equally important and powerful and are used for different purposes. It should be noted here that although spatial and frequency domain techniques are used differently, they are both related. For example, suppose a spatial filter is used to reduce noise in an image. As a result of this filter, noise level in the image will be reduced, but at the same time, the frequency spectrum of the image will also be affected due to this reduction in noise.

The following sections discuss some fundamental issues about frequency and spatial domains. This discussion, although general, will help us throughout the entire chapter.

9.3 Fourier Transform and Frequency Content of a Signal

As you may remember from your mathematics or other courses, any periodic signal may be decomposed into a collection of sines and cosines of different amplitudes and frequencies, called Fourier series, as follows:

$$f(t) = \frac{a_0}{2} + \sum_{n=1}^{\infty} a_n \cos n\omega t + \sum_{n=1}^{\infty} b_n \sin n\omega t \quad (9.1)$$

When these sines and cosines are added together, the original signal is reconstructed. This conversion to frequency domain is called Fourier series, and the collection of different frequencies present in the equation is called *frequency spectrum* or *frequency content* of the signal. Of course, although the signal is in the amplitude-time domain, the frequency spectrum is in the amplitude-frequency domain. To understand this better, let's look at an example.

Consider a signal in the form of a simple sine function like $f(t) = \sin(t)$. Since this signal consists only of one frequency and a constant amplitude, the frequency spectrum representing it consists of a single value at the given frequency, as shown in Figure 9.3. Obviously, if we plot the function represented by the arrow in Figure 9.3(b) with the given frequency and amplitude, we will have the same sine function reconstructed. The plots in Figure 9.4 are similar and represent $f(t) = \sum_{n=1,3,\dots,15} \frac{1}{n} \sin(nt)$. The frequencies are

size, orientation, and other properties of objects in images, and extraction of depth information. Some techniques may be used for multiple purposes, as we will see later. For example, moment equations may be used for object recognition as well as calculation of position and orientation of objects.

Generally, it is assumed that image processing routines have already been applied to the image or that they are available for further use when needed to improve and prepare the image for analysis. Image analysis routines and techniques may be used on both binary and gray images. In the following sections, some of these techniques are discussed.

9.21 Object Recognition by Features

Objects in an image may be recognized by their features. These features may include, but are not limited to: gray level histograms; morphological features such as area, perimeter, number of holes, and others; eccentricity; cord length; and moments. In many cases, the information extracted is compared to *a priori* information about the object, which may be in a look-up table. For example, suppose two objects are present in the image, one with two holes and one with one hole. Using previously discussed routines, it is possible to determine how many holes each part has and, by comparing the two parts (let's say they are assigned regions 1 and 2) to a look-up table, it is possible to determine what each of the two parts are. In another example, assume a moment analysis is performed for a known object and the moment, relative to an axis, is calculated at many angles and the data is stored in a look-up table. Later, when the moment of the part in the image is calculated relative to the same axis and is compared to the look-up table, the angle of the part in the image can be estimated.

The following is a discussion of a few techniques and different features that may be used for object recognition.

9.21.1 Basic Features Used for Object Identification

The following morphological features may be used for object recognition and identification:

- (a) **Gray Levels:** Average, maximum, or minimum gray levels may be used to identify different parts or objects in an image. As an example, assume there are three parts in an image, each one with a different color or texture. The colors and textures will create different gray levels in the image. If the average, maximum, or minimum gray levels of the objects are found (e.g., through histograms mapping), the objects can be recognized by comparison of this information. In other cases, even the presence of one particular gray level may be enough to recognize a part.
- (b) **Perimeter, area, diameter, number of holes, and other similar morphological characteristics** may be used for object identification. The perimeter of an object may be found by first applying an edge detection routine, and then, by counting the number of pixels on the perimeter. The Left-Right search technique of section 9.13 can also be used to calculate the perimeter by counting the pixels that are on the path in an accumulator. Area can be calculated by region growing techniques. Moment equations can also be used, as will be discussed later. Diameter for noncircular objects

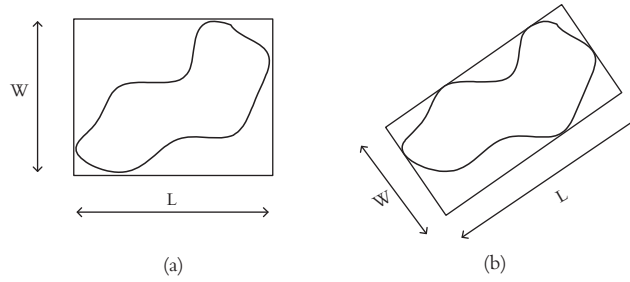


Figure 9.62 (a) Aspect ratio of an object, (b) minimum aspect ratio.

is the maximum distance between any two points on any line that crosses the identified area of the object.

(c) **Aspect Ratio:** Aspect ratio is the width to length ratio of an enclosing rectangle about the object, as shown in Figure 9.62. All aspect ratios are sensitive to orientation, except the minimum aspect ratio. Therefore, the minimum aspect ratio is usually used to identify objects.

(d) **Thinness** is defined as one of the two following ratios:

$$1. \text{ Thinness} = \frac{(\text{perimeter})^2}{\text{area}} \quad (9.21)$$

$$2. \text{ Thinness} = \frac{\text{diameter}}{\text{area}} \quad (9.22)$$

(e) **Moments:** Due to their importance, moments are discussed in the next section.

9.21.2 Moments

The moment of an object within an image is:

$$M_{a,b} = \sum_{x,y} x^a y^b I_{x,y} \quad (9.23)$$

where $M_{a,b}$ is the moment of the object with a and b indices, x and y are the coordinates of each pixel raised to the power of a and b , and $I_{x,y}$ is the intensity of the pixel, as in Figure 9.63. If the image is binary, the intensities are either 1 (or on) for the object and 0 (off) for the background, and therefore, only the pixels that are turned on are considered. In gray images, the intensities may vary greatly, and consequently, the value of the moment may be exceedingly influenced by gray values. As a result, although it is mathematically possible to apply moment equations to a gray image, it is not practical or useful unless other rules are engaged, too. For binary images, $I_{x,y}$ is either 0 or 1; therefore, considering only the on-pixels in the image, Equation (9.23) simplifies to:

$$M_{a,b} = \sum_{x,y} x^a y^b \quad (9.24)$$

To calculate the moments, first determine whether or not each pixel belongs to the object (is turned on); if so, raise the coordinates of the location of the pixel to the given

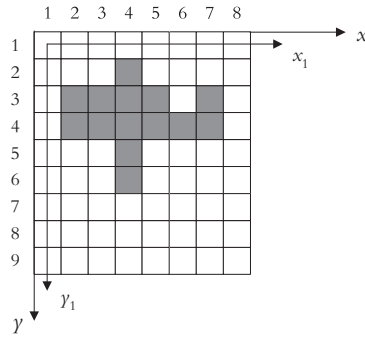


Figure 9.63 Calculation of the moment of an image. For each pixel that belongs to the object, the coordinates of the pixel are raised to the powers indicated by the moment's indices. The summation of the values thus calculated will be the particular moment of the image.

values of a and b . The summation of this operation over the entire image is the particular moment of the object with a and b indices.

$M_{0,0}$ is the moment of the object with $a = 0$ and $b = 0$. This means the x and y coordinate values of all on-pixels are raised to a power of 0. $M_{0,2}$ means all x values are raised to the power of 0 and all y values are raised to the power of 2, and so on. All combinations of values between 0 and 3 are common.

Distances x and y are measured either from a fictitious coordinate frame located at the edge of the image (x, y) or are measured from a coordinate frame formed by the first row and column of the image. Since the distances are measured by counting the number of pixels, the use of the first row and column as the coordinate frame is more logical. However, note that in this case, all distances should be measured to the centerline of the pixel row or column. As an example, the first on-pixel on the second row is pixel (2,4). The x distance of the pixel from the x_1 - y_1 coordinate frame will be 3, whereas the same coordinate from the x - y coordinate is 4 (or more accurately, 3.5 pixels). As long as the same distances are used consistently, the choice is not important.

Based on the above, since all numbers raised to the power of 0 are equal to 1, all x^0 and y^0 's are equal to 1. Therefore, the $M_{0,0}$ moment is the summation of all on-pixels, which is the area of the object. This moment can be used to determine the nature of an object and to distinguish it from others that have a different area. Obviously, the $M_{0,0}$ moment can also be used to calculate the area of an object within an image.

Similarly, $M_{0,1}$ is $\sum x^0 y^1$ for all on-pixels, or the summation of $1 \times y$ values, which is the summation of the y -coordinates of all on-pixels from the x -axis. This is similar to the first moment of the area relative to the x -axis. Therefore, the location of the center of the area relative to the x -axis can be calculated by:

$$\bar{y} = \frac{\sum y}{\text{area}} = \frac{M_{0,1}}{M_{0,0}} \quad (9.25)$$

So, by simply dividing the two moments, you may calculate the $\bar{y}$ coordinate of the center of the area of the object. Similarly, the location of the center of the area relative to

the y -axis is:

$$\bar{x} = \frac{\sum x}{\text{area}} = \frac{M_{1,0}}{M_{0,0}} \quad (9.26)$$

This way, an object may be located within an image regardless of its orientation (the orientation will not change the location of the center of an area). Of course, this information can be used to locate an object, say, for grabbing by a robot.

$M_{0,2}$ is $\sum x^0 y^2$ and represents the second moment of the area relative to the x -axis. Similarly, $M_{2,0}$ is the second moment of the area relative to the y -axis. As you can imagine, the moment of inertia of an object such as the one in Figure 9.63 varies significantly as the object rotates about its center. Suppose we calculate the moments of the area about an axis, say, the x -axis, at different orientations. Since each orientation creates a unique value, a look-up table that contains these values can later be used to identify the orientation of the object. Therefore, if a look-up table is prepared containing the values of the moments of inertia of the known object at different orientations, the subsequent orientation of the object can be estimated by comparing its second moment to the values in the look-up table. Of course, if the object translates within an image, its moments of inertia will also change, rendering it impossible to determine the orientation except in known locations. However, with a simple application of the parallel axes theorem, the second moments about the center of the area can be calculated. Since this measure is independent of the location, it can be used to determine the orientation of the object regardless of its location. Therefore, using the moment equations, the object, its location, and its orientation can be identified. In addition to identification of the part, the information can be used in conjunction with a robot controller to direct the robot to pick up the part and/or operate on it.

Other moments can also be used similarly. For example, $M_{1,1}$ represents the product of inertia of the area and can also be used for object identification. Higher-order moments such as $M_{0,3}$, $M_{3,0}$, $M_{1,2}$, and so on can also be used to identify objects and their orientation. Imagine two objects relatively similar in shape, as in Figure 9.64(a). It is possible that the second moments, areas, perimeters, or other morphological characteristics of the two objects may be similar or close to each other such that they may not be useful in object identification. In this case, a small difference between the two objects may be exaggerated through higher-order moments, making object identification possible. The same is true for an object with a small asymmetry (Figure 9.64(b)). The orientation of the object may be found by higher-order moments.

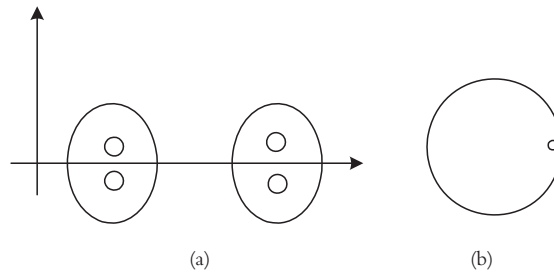


Figure 9.64 Small differences between objects or small asymmetry in an object may be detected using higher-order moments.

A moment invariant is a measure of an object based on its different moments, and is independent of its location, orientation, and scale factor. Therefore, the moment invariants may be used for object recognition and parts identification without regard to camera set-up, location, or orientation. There are seven different moment invariants, such as:

$$MI_1 = \frac{M_{0,0}M_{2,0} - M_{1,0}^2 + M_{0,0}M_{0,2} - M_{0,1}^2}{M_{0,0}^3} \quad (9.27)$$

Refer to Reference [2] for the other six moment invariants.

Example 9.11

For the simple object in a low-resolution image of Figure 9.65, calculate the area, center of the area, and second moments of area of the object relative to the x_1 , y_1 -axes.

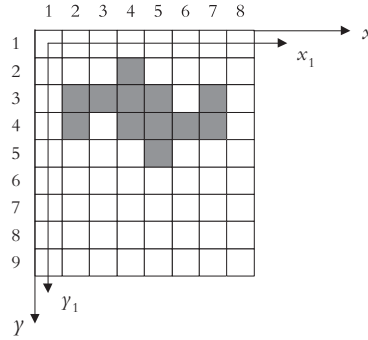


Figure 9.65 Image used for Example 9.11.

Solution: Measuring the distances of each on-pixel from the x_1 , y_1 -axes and substituting the measurements into the moment equations will yield the following results:

$$M_{0,0} = \sum x^0 y^0 = 12(1) = 12$$

$$M_{1,0} = \sum x^1 y^0 = \sum x = 2(1) + 1(2) + 3(3) + 3(4) + 1(5) + 2(6) = 42$$

$$M_{0,1} = \sum x^0 y^1 = \sum y = 1(1) + 5(2) + 5(3) + 1(4) = 30$$

$$\bar{x} = \frac{M_{1,0}}{M_{0,0}} = \frac{42}{12} = 3.5 \quad \text{and} \quad \bar{y} = \frac{M_{0,1}}{M_{0,0}} = \frac{30}{12} = 2.5$$

$$M_{2,0} = \sum x^2 y^0 = \sum x^2 = 2(1)^2 + 1(2)^2 + 3(3)^2 + 3(4)^2 + 1(5)^2 + 2(6)^2 = 178$$

$$M_{0,2} = \sum x^0 y^2 = \sum y^2 = 1(1)^2 + 5(2)^2 + 5(3)^2 + 1(4)^2 = 82$$

The same procedure may be used for an image with much higher resolution. There will just be many more pixels to deal with. However, a computer program can handle as many pixels as necessary without difficulty. ■

Example 9.12

In a certain application, a vision system looks at an 8×8 binary image of rectangles and squares. The squares are either 3×3 -pixel solids, or 4×4 hollow, while the rectangles are 3×4 solids. Through guides, jigs, and brackets, we can be certain the objects are always parallel to the reference axes, as shown in Figure 9.66, and that

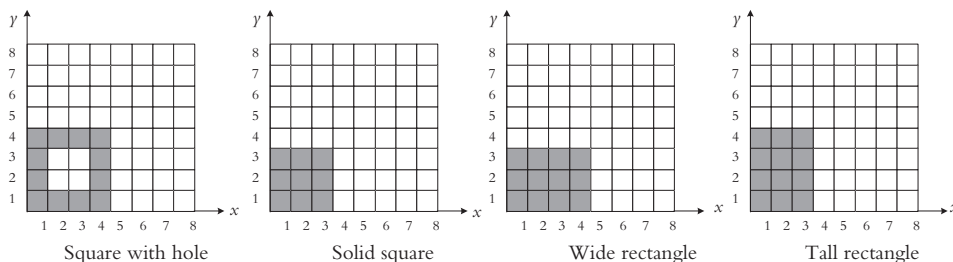


Figure 9.66 Image used for Example 9.12.

the lower left corners of the objects are always at pixel 1,1. We want to only use the moment equations to distinguish the parts from each other. Find one set of lowest values a and b in the moment equation that would be able to do so, with corresponding values for each part. For this example, use the absolute coordinates of each pixel for distances from the corresponding axes.

Solution: Using the moment equations, we will calculate the different moments for all four until we find one set that are all unique for each object.

Square with hole	Solid square	Wide rectangle	Tall rectangle
$M_{0,0} = 12$	$M_{0,0} = 9$	$M_{0,0} = 12$	$M_{0,0} = 12$
$M_{0,1} = 30$	$M_{0,1} = 18$	$M_{0,1} = 24$	$M_{0,1} = 30$
$M_{1,0} = 30$	$M_{1,0} = 18$	$M_{1,0} = 30$	$M_{1,0} = 24$
$M_{1,1} = 75$	$M_{1,1} = 36$	$M_{1,1} = 60$	$M_{1,1} = 60$
$M_{0,2} = 94$	$M_{0,2} = 42$	$M_{0,2} = 56$	$M_{0,2} = 90$

As shown, the lowest set of moment indices that yields a unique solution for each object is $M_{0,2}$. Of course, $M_{2,0}$ would result in similar numbers. ■

Example 9.13

For the image of the screw in Figure 9.67, calculate the area, $\bar{x}$, $\bar{y}$, $M_{0,2}$, $M_{2,0}$, $M_{1,1}$, $M_{2,0} @ \bar{x}$, $M_{0,2} @ \bar{y}$ and the moment invariant.

Solution: A macro program called moments.macro was written for the OptimasTM 6.2 vision software to calculate the moments. In this program, distances used for moments are all in terms of the number of pixels and not in units of length. The values were calculated for five separate cases, horizontal, 30° , 45° , 60° , and vertical. Small variations in the results are due to rotation operations. Every time a part of an image is rotated, since every point in the image must be converted with a sine or cosine function, it changes slightly. Otherwise, as you can see, the results are consistent. For example, as



Figure 9.67 Image used for Example 9.13. (© Saeed Niku.)

the part is rotated in place, the location of its center of area does not change. You also can see that the moment invariant is constant and, from the moment of inertia information about the centroid, the orientation can be estimated. This information can now be used to identify the object or to direct a robot controller to send the robot arm to the location, with proper orientation, to pick up the part.

	Horizontal	30°	45°	60°	Vertical
<i>Area</i>	3713	3747	3772	3724	3713
<i>x-bar</i>	127	123	121	118	113
<i>y-bar</i>	102	105	106	106	104
$M_{0,2}$	38.8 E6	43.6 E6	46.4 E6	47.6 E6	47.8 E6
$M_{2,0}$	67.6 E6	62.6 E6	59 E6	53.9 E6	47.8 E6
$M_{1,1}$	48.1 E6	51.8 E6	52 E6	49.75 E6	43.75 E6
<i>Moment Invariant</i>	7.48	7.5	7.4	7.3	7.48
$M_{2,0}@{\bar{x}}$	7.5 E6	5.7 E6	3.94 E6	2.07 E6	0.264 E6
$M_{0,2}@{\bar{y}}$	0.264 E6	2.09 E6	3.77 E6	5.7 E6	7.5 E6

The following is the listing of the moments.macro program, written for Optimas. Although this program cannot directly be used with other software, it is listed here to show how simply a program can be developed to do similar operations. The Excel part of the program is nothing more than a simple set of Excel equations that operate on the coordinates of all pixels, and later, are summed up.

*/*MOMENTS.MAC PROGRAM* Written by Saeed Niku, Copyright 1998

This macro checks an active image within Optimas vision system. It records the coordinates of all pixels above the given threshold. It subsequently writes the coordinates into an Excel worksheet, which determines the moments. Moments.mac will then read back the data and display it. The DDE commands communicate the data between Excel and Optimas macro program. If your number of coordinates is more than 20,000 pixels, you must change the command below. */

```

BinaryArray = GetPixelRect (ConvertCalibToPixels(ROI));
INTEGER NewArray[,]; Real MyArea; Real Xbar; Real Ybar;
Real Mymoment02; Real Mymoment20; Real Mymoment11; Real VariantM;
Real MyMXBar; Real MyMYBar;

For(Xcoordinate = 0; Xcoordinate<= (VectorLength(BinaryArray[0,])-1);
  Xcoordinate++)
{
  For(Ycoordinate = 0; Ycoordinate <= (VectorLength(BinaryArray[,0])-1);
    Ycoordinate++)
  {
    If (BinaryArray[Ycoordinate,Xcoordinate] > 100)
    {
      NewArray ::= Xcoordinate : Ycoordinate;
    } } }
hChanSheet1 = DDEInitiate ("Excel","Sheet1");
DDEPoke(hChanSheet1,"R1C1:R2000C2",NewArray);
DDETerminate(hChanSheet1);

Show("Please Enter to Show Values");
hChanSheet1 = DDEInitiate ("Excel","Sheet1");
DDERequest(hChanSheet1,"R1C14",MyArea);
DDERequest(hChanSheet1,"R2C14",Ybar);
DDERequest(hChanSheet1,"R3C14",Xbar);
DDERequest(hChanSheet1,"R4C14",Mymoment02);
DDERequest(hChanSheet1,"R5C14",Mymoment20);
DDERequest(hChanSheet1,"R6C14",Mymoment11);
DDETerminate(hChanSheet1);

VariantM=(MyArea*Mymoment20*1000000.0-Xbar*Xbar
          +MyArea*Mymoment02*1000000.0-Ybar*Ybar)
          /(MyArea*MyArea*MyArea);

MyMYBar=(Mymoment20*1000000.0-MyArea*Xbar*Xbar)/1000000.0;
MyMXBar=(Mymoment02*1000000.0-MyArea*Ybar*Ybar)/1000000.0;

MacroMessage("Area=",MyArea,"\n","Xbar=",Xbar,"\n",
             "Ybar=",Ybar,"\n","Moment02=",Mymoment02," x10^6",
             "\n","Moment20=",Mymoment20," x10^6","\n",
             "Moment11=",Mymoment11," x10^6","\n","Invariant 1="
             ,VariantM);
MacroMessage("Moment20@Xbar=",MyMXBar," x10^6","\n",
             "Moment02@Ybar=",MyMYBar," x10^6");

```

9.21.3 Template Matching

Another technique for object recognition is model or template matching. If a suitable line drawing of the scene is found, the topological or structural elements such as total number of lines (sides), vertices, and interconnections can be matched to a model. Coordinate transformations such as rotation, translation, and scaling can be performed to eliminate

the differences between the model and the object resulted from position, orientation, or depth differences between them. This technique is limited by the fact that *a priori* knowledge of the object models is needed for matching. Therefore, if the object is different from the models, they will not match and the object will not be recognized. Another major limitation is that if one object is occluded by other objects, it will not match a model.

9.21.4 Discrete Fourier Descriptors

Similar to a Fourier transform calculated for an analog signal, a Discrete Fourier Transform (DFT) of a set of discrete points (such as pixels) can also be calculated. This means that if the contour of an object within an image is found (such as in edge detection), the discrete pixels of the contour can also be used for DFT calculations. The result of DFT calculation is a set of frequencies and amplitudes in frequency domain that describe the spatial relationship of the points in question.¹⁴

To calculate the DFT of a set of points in a plane, assume the plane is a real-imaginary plane, such that each point is described by an $x + iy$ relationship. If the contour is completely traced around, starting from any pixel, and the locations of the points are measured, the information can be used to calculate the corresponding frequency spectrum of the set. Matching these frequencies with the frequencies found for possible objects in a look-up table may be used to determine the nature of the object. In one unpublished experiment, matching 8 frequencies yielded enough information about the nature of the object (an airplane). Matching 16 frequencies could determine the type of an airplane from a large class of planes. An advantage of this technique is that the Fourier transform can very simply be normalized for size, position, and orientation. A disadvantage of the technique is that it requires a complete contour of the object. Of course, other techniques, such as the Hough transform, can be used to complete broken contours of the object.

9.21.5 Computed Tomography (CT)

Tomography is a technique of determining the distribution of material density in the examined part. In computed tomography (CT), a three-dimensional image of the object's density distribution is reconstructed from a large number of two-dimensional images of the density taken by different scanning techniques such as X-rays or ultrasonics. In computed tomography, it is assumed that the part consists of a sequence of overlaying slices. Images of density distribution of each slice are taken repeatedly around the object. Although partial coverage of the part has been used as well, a complete coverage of 360° is preferred. The data is stored in a computer, and subsequently, is reduced to a three-dimensional image of the part's density distribution that is shown on a monitor.

Although this technique is completely different from the other techniques discussed above, it is a viable technique for object recognition. In many situations, either alone or in conjunction with other techniques, computed tomography may be the only way to recognize an object or to differentiate it from other similar objects. Specifically, in medical situations, CT scan can be used in conjunction with medical robots where the three-dimensional mapping of the internal organs of the human body may be used to direct the robot for surgical operations.

9.22 Depth Measurement with Vision Systems

Extracting depth information from a scene is performed using two basic techniques. One is the use of range finders in conjunction with a vision system and image-processing techniques. In this combination, the scene is analyzed in relation to the information gathered by range finders about the distances of different portions of an environment or the location of particular objects or sections of the object. Second is the use of binocular or stereo vision similar to humans and animals. In this technique, either simultaneous images from multiple cameras or multiple images from one camera that moves on a track are used to extract depth information. As long as the scene does not change during this operation, the results will be the same as the use of multiple cameras. Since the location of the multiple (usually two) cameras in relation to any particular point in the scene is slightly different, each camera develops a slightly different image. By analyzing and measuring the differences between the two scenes, depth information can be extracted.

9.22.1 Scene Analysis versus Mapping

Scene analysis refers to the analysis of images developed by a camera or other similar devices in which a complete scene is analyzed. In other words, the image is a complete replica of the scene within the resolution limit of the device where all the details of the scene are included in the image. In this case, more processing is generally required to extract information from the image, but more information can be extracted. For instance, in order to identify an object within a scene, the image may have to be filtered and enhanced, segmented by edge detection or thresholding, the part isolated by region growing, and then identified by extracting its features and comparing them to a template or look-up table. On the other hand, mapping refers to drawing the surface topology of a scene or object where the image consists of a set of discrete distance measurements, usually at low resolutions. The final image is a collection of lines that relate to the relative position of points on the object at discrete locations. Since the image is already sliced, less processing is required in analysis of mapped images, but less information can be extracted from the scene as well. Each technique has its own merits, benefits, and limitations and is used for different purposes, including navigation.

9.22.2 Range Detection and Depth Analysis

Range measurement and depth analysis is performed using many different techniques such as active ranging,²⁰ stereo imaging, scene analysis, or specialized lighting. Humans use a combination of techniques to extract information about the depth and positional relationship between different elements of an image. Even in a two-dimensional image, humans can extract useful information using details such as the changing size of similar elements, vanishing lines, shadows, and changing intensity of textures and shades. Since many artificial intelligence techniques are based on, and studied for, understanding of the way humans do things, a number of depth measurement techniques are designed after similar human operations.¹⁵

9.22.3 Stereo Imaging

An image is the projection of a scene into the image plane through an ideal lens. Therefore, every point in the image will correspond to a certain point in the scene.

However, the depth information of the point is lost in this projection and cannot simply be retrieved from a single image. If two images of the same scene are taken, the relative depth of different points from the image plane can be extracted by comparing the two images; the differences represent the spatial relationship between different points.^{16,17} Humans do the same, automatically, by combining the two images and forming a three-dimensional image.^{18,19,21} The stereo image used for depth measurement is considered a 2.5-dimensional image. Many more images are required to form a true three-dimensional image.

Depth measurement using stereo images requires two operations:

1. To determine the point-pairs in the two images that correspond to the same point in the scene. This is called correspondence or disparity of the point-pair. This is a difficult operation since some points in one image may not be visible in another, or since due to perspective distortion, sizes and spatial relationships may be different in the two images.
2. To determine the depth or location of the point on the object or in the scene by triangulation or other techniques.

Generally, if the two cameras (or the relative locations of a single camera used twice to get two images of a nonmoving, static scene) are accurately calibrated, triangulation is relatively simple as long as enough corresponding points have been found.

Correspondence points can be determined by matching specific features, such as corners or small segments, from the two images. Depending on their locations, correspondence points can create matching problems. Consider the two marks A and B in Figure 9.68. In each case, the two cameras will see the marks as shown in (a) and (b). Although the locations of the two marks are different, the cameras will see them similarly. As a result, the marks may be located wrongly (unless additional information such as vanishing lengths are also considered).

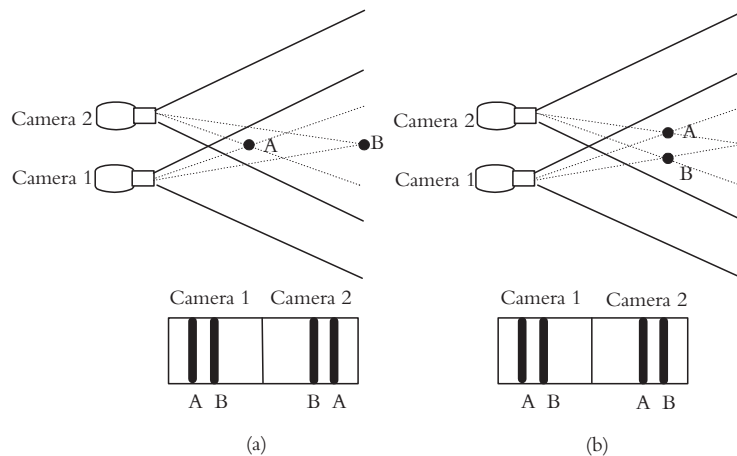


Figure 9.68 Correspondence problem in stereo imaging.

to do this is to take the average values of the three files for the same pixel location and to use that as a substitute for the gray value. Therefore, the histogram of a gray image shows the same exact values for all three channels of RGB. For more information about the image processing of colored images please refer to other resources such as References 2 and 13.

9.26 Heuristics

Heuristics is a collection of rules of thumb developed for semi-intelligent systems in order to enable them to select a predetermined decision from a list based on the current situation. Heuristics is used in conjunction with mobile robots, but has applications in many fields.

Consider a mobile robot that is supposed to navigate through a maze. Imagine the robot starts at a point and is equipped with a sensor that alerts its controller that the robot has reached an obstacle such as a wall. At this point, the controller has to decide what to do next. Let's say the first rule is that when encountering an obstacle, the robot should turn left. As the robot continues, it may reach another wall, turn left again, and continue. Suppose that after three left turns, the robot reaches the starting point. In this case, should it continue to turn left? Obviously, this will result in a never-ending loop. The second rule may be to turn right if the first point is encountered. Now imagine that after a left turn, the robot gets to a dead end. Then what? A third rule may be to trace back the path until an alternate route can be found. As you can see, there are many different situations the robot may encounter. Each one of these situations must be considered by the designer, and a decision must be provided. The collection of these rules is the heuristics rule base for the controller to "intelligently" decide how to control the motions of the robot. However, it is important to realize that this is not true intelligence since the controller is not really making decisions but merely selecting from a set of decisions that have already been made. If a new situation is encountered that is not in the rules base, the controller will not know how to respond.²⁶

9.27 Applications of Vision Systems

Vision systems may be used for many different applications, including in conjunction with robotic operations and robots. Vision systems are commonly used for operations that require information from the work environment and include inspection, navigation, part identification, assembly operations, surveillance, control, and communication.

Suppose that in an automatic manufacturing setting, a circuit board is to be manufactured. One important part in this operation is the inspection of the board at different states before and after certain operations. A common method is to set up a cell where an image of the part is taken and subsequently modified, improved, and altered. The processed image is compared to a look-up image. If there is a match, the part is accepted. Otherwise, either the part is rejected or is repaired. These image-processing and analysis operations are generally made up of the processes discussed earlier. Most commercial vision systems have embedded routines that can be called from a macro program, making it very easy to set up a system.

Vision systems have been used for many applications, for example, for locating radioactive pucks,²⁷ random bin picking,²⁸ creating an automated brake inspection system,²⁹ measuring robotic motions and external objects,³⁰ food inspection such as texture of cookies and consistency of packaging,³¹ creating adaptive behavior for mobile robots,³² analyzing the health of agricultural crops,³³ and many others.

In navigation, the scene is usually analyzed for finding acceptable pathways, obstacles, and other elements that confront the robot.³⁴ In some operations, the vision system sends its information to an operator, who controls the motions from a distance. This is very common in telerobotics as well as in space applications.³⁵ In some medical applications, the surgeon guides the device, whether a surgical robot or a small investigative, exploratory device such as an angiogram, through its operations.³⁶ Autonomous navigation requires the integration of depth measurement with the vision system, either by stereo vision analysis, or by range finders. It also requires heuristic rules of behavior for the robotic device to navigate around an environment.

In another application,^{37,38} an inexpensive laser diode was mounted next to a camera. The projected laser light was captured by the camera and was used to both measure the depth of a scene as well as to calibrate the camera. In both cases, due to the brightness of the laser light and bleeding effects, the image contained a large, bright circular spot. To identify the dot and separate it from the rest of the scene, a histogram and thresholding operation was used. Subsequently, the circle was identified and skeletonized until only the center of the circle remained. The location of the pixel representing the center of the circle was then used in a triangulation method to calculate the depth of the image, or to calibrate the camera.

These simple examples are all related to what we have discussed. Although many other routines are available, the fundamental knowledge about vision systems enables you to proceed with an application and adapt to your application what vision systems have to offer.

9.28 Design Project

There are many inexpensive digital cameras on the market that can be used to create a simple vision system. They are simple, small, and lightweight and provide a simple image that can be captured by computers and be used to develop a vision system. In fact, many cameras come with the software to capture and digitize an image. Standard still and video cameras can also be used for capturing images. In this case, although you can capture an image for later analysis, due to the additional steps of downloading the image from the camera to the computer, the image is not available for immediate use.

Additionally, many simple programs such as Adobe's PhotoshopTM have many routines similar to what we have discussed in this chapter. Additional routines may be developed using common computer languages such as C. Many other routines may be downloaded from the public domain. The final product will be a simple vision system that can be used to perform vision tasks. This may be done independently, or in conjunction with your 3-axis robot, and can include routines for parts identification and pick up, the development of mobile robots, and many other similar devices.

Most images shown in this chapter were captured and processed by standard digital cameras and the vision systems in the Mechanical Engineering Robotics laboratory at Cal Poly, including MVS909TM and OptimasTM 6.2 vision systems and PhotoshopTM.